



## 12 C6 Thuật TOÁN NHAN DANG Mang CNN

HeThongMayTinh (Trường Đại học Công nghiệp Thành phố Hồ Chí Minh)



Scan to open on Studeersnel

# DIGITAL IMAGE PROCESSING (XỬ LÝ ẢNH SỐ)

Chương: 6

## NHẬN DẠNG ĐỐI TƯỢNG

ThS: TRẦN VĂN HÙNG

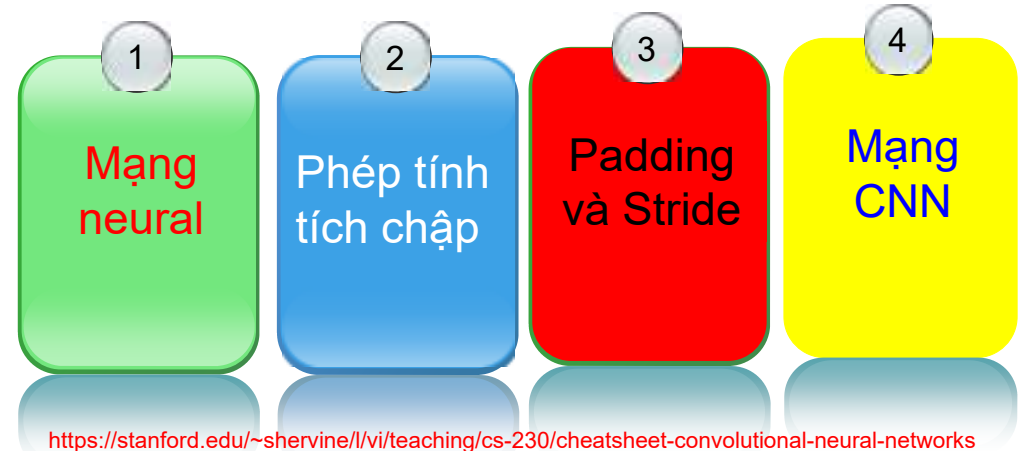
Email: tranvanhung@iuh.edu.vn

E-Learning: Trần Văn Hùng

ocwfet.iuh.edu.vn



# ỨNG DỤNG DEEPLARNING DÙNG CNN



## Phép tính convolution (tích chập)

www.tvhungdhcn@gmail

- ❖ Convolutional Neural Networks (CNN) là một trong những mô hình deep learning phổ biến nhất và có ảnh hưởng nhiều nhất trong thị giác máy tính. CNN được dùng trong nhiều bài toán như nhận dạng ảnh, phân tích video, ảnh MRI, hoặc cho các bài toán trong lĩnh vực xử lý ngôn ngữ tự nhiên.



Chúng ta nhìn thấy

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 107 | 48  | 200 | 86  | 3   | 78  | 228 | 66  | 207 | 32  |
| 158 | 56  | 94  | 218 | 126 | 97  | 90  | 185 | 63  | 88  |
| 108 | 201 | 200 | 189 | 93  | 5   | 4   | 120 | 48  | 200 |
| 188 | 132 | 202 | 121 | 119 | 230 | 40  | 238 | 76  | 149 |
| 180 | 65  | 31  | 138 | 98  | 226 | 164 | 32  | 200 | 16  |
| 134 | 225 | 229 | 180 | 141 | 155 | 152 | 188 | 88  | 252 |
| 86  | 17  | 238 | 223 | 34  | 220 | 64  | 187 | 187 | 210 |
| 312 | 244 | 86  | 30  | 100 | 180 | 95  | 188 | 38  | 111 |
| 208 | 221 | 281 | 223 | 181 | 179 | 118 | 184 | 8   | 88  |
| 204 | 271 | 1   | 187 | 137 | 148 | 80  | 38  | 101 | 132 |

Máy tính thấy

This document is beschikbaar op

Downloaded by: Quan Dang (dangquan3172006@gmail.com)

## Phép tính convolution (tích chập)

www.tvhungdhcn@gmail

- ❖ Tích chập là giai đoạn đầu để trích xuất các đặc trưng từ ảnh vào. Tích chập duy trì mối quan hệ giữa các pixel bằng cách xác định các đặc trưng ảnh và sử dụng các vùng nhỏ của dữ liệu đầu vào. Nó là phép toán có 2 đầu vào như ma trận ảnh và 1 bộ lọc hay còn gọi là kernel.
- ❖ Một ma trận ảnh vào (volume) được xác định ( $H \times W \times D$ )
- ❖ Một bộ lọc hay kernel ( $f_H \times f_W \times D$ )
- ❖ Ngõ ra ma trận ảnh được xác định:  $(H - f_H + 1) \times (W - f_W + 1) \times 1$

$H \times W \times D$  \*  $f_H \times f_W \times D$  =  $(H - f_H + 1) \times (W - f_W + 1) \times 1$



## Phép tính convolution (tích chập)

www.tvhungdhn@gmail

- Xét 1 ma trận 5x5 có giá trị pixel là 0 và 1. Ma trận kernel 3x3 như hình sau

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

\*

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

5 x 5 - Image Matrix

3 x 3 - Filter Matrix

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

Image

|   |  |  |
|---|--|--|
| 4 |  |  |
|   |  |  |
|   |  |  |

Convolved Feature

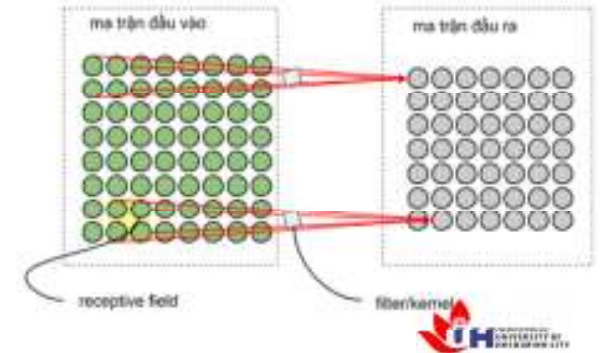
- Vậy lớp tích chập của ma trận ảnh 5x5 nhân với ma trận kernel 3x3 gọi là 'Feature Map' như hình.



## Phép tính convolution (tích chập)

www.tvhungdhn@gmail

- Ví dụ ảnh xám được biểu diễn dưới dạng ma trận A kích thước m\*n.
- Ta định nghĩa **kernel** là một ma trận vuông kích thước k\*k trong đó k là số lẻ. k có thể bằng 1, 3, 5, 7, 9,... Ví dụ kernel kích thước 3\*3

$$W = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$


## Phép tính convolution

www.tvhungdhn@gmail

- Kí hiệu phép tính convolution ( $\otimes$ ), ký hiệu:  $Y = X \otimes W$

Với mỗi phần tử  $x_{i,j}$  trong ma trận X lấy ra một ma trận có kích thước bằng kích thước của kernel W có phần tử  $x_{i,j}$  làm tâm (đây là lý do vì sao kích thước của kernel thường lẻ) gọi là ma trận A. Sau đó tính tổng các phần tử của phép tính element-wise của ma trận A và ma trận W, rồi điền vào ma trận kết quả Y.



## Phép tính convolution

www.tvhungdhn@gmail

- Kí hiệu phép tính convolution ( $\otimes$ ), ký hiệu:  $Y = X \otimes W$

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

 $\otimes$ 

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

 $=$ 

|   |  |  |
|---|--|--|
| ? |  |  |
|   |  |  |
|   |  |  |

X

W

Y

Ví dụ khi tính tại  $x_{2,2}$  (ô khoanh đỏ trong hình), ma trận A cùng kích thước với W, có  $x_{2,2}$  làm tâm có màu nền là màu cam như trong hình. Sau đó tính

$$y_{1,1} = \text{sum}(A \otimes W)$$



❖ Kí hiệu phép tính convolution ( $\otimes$ ), ký hiệu:  $Y=X \otimes W$

$$y_{1,1} = \text{sum}(A \otimes W)$$

$$= x_{11} * w_{11} + x_{12} * w_{12} + x_{13} * w_{13} + x_{21} * w_{21} + x_{22} * w_{22} + x_{23} * w_{23} + x_{31} * w_{31} + x_{32} * w_{32} + x_{33} * w_{33}$$

= 4. Và làm tương tự với các phần tử còn lại trong ma trận.

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

$X \otimes W$

|   |  |  |
|---|--|--|
| 4 |  |  |
|   |  |  |
|   |  |  |

Y



❖ Kí hiệu phép tính convolution ( $\otimes$ ), ký hiệu:  $Y=X \otimes W$

Vấn đề là phải xử lý như thế nào với phần tử ở viền ngoài như  $x_{1,1}$ . Bình thường khi tính thì sẽ bỏ qua các phần tử ở viền ngoài, vì không tìm được ma trận A ở trong X.

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |



❖ Kí hiệu convolution ( $\otimes$ ), ký hiệu phép tính:  $Y=X \otimes W$

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

$X \otimes W$

|   |   |   |
|---|---|---|
| 4 | 3 | 4 |
| 2 | 4 | 3 |
| 2 | 3 | 4 |

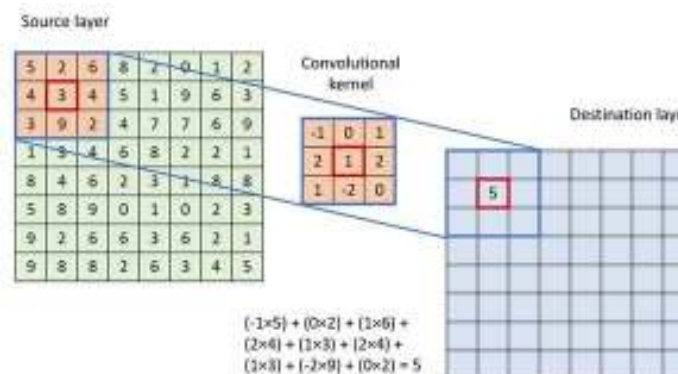
Y

Nên ta để ý thấy ma trận Y có kích thước nhỏ hơn ma trận X. Kích thước của ma trận Y là  $(m-k+1) * (n-k+1)$

Dit document is beschikbaar op



❖ Kích thước kernel của tầng convolution hầu hết đều là số lẻ, ví dụ như 3x3 hay 5x5. Với kích thước filter lẻ, các giá trị của feature map sẽ xác định một tâm điểm ở tầng phía trước.



Nếu ta chọn kernel có kích thước 2x2, 4x4 thì chúng ta sẽ gặp khó khăn khi muốn tìm vị trí tương ứng của các giá trị feature map trên không gian ảnh.

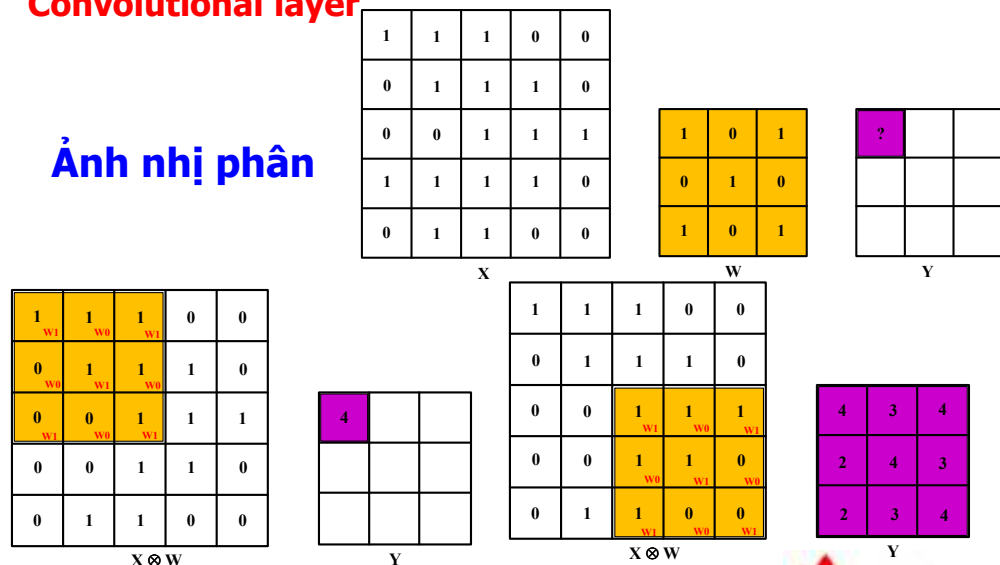


## Phép tính convolution

www.tvhungdhn@gmail

### Convolutional layer

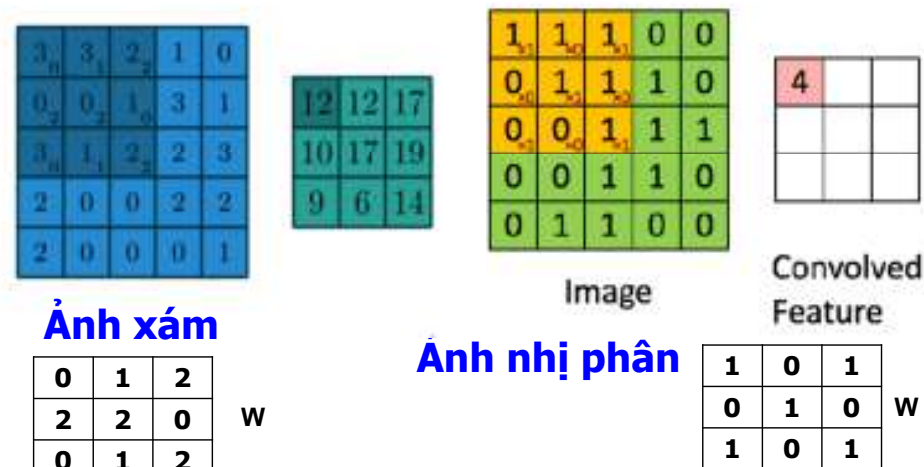
#### Ảnh nhị phân



## Phép tính convolution

www.tvhungdhn@gmail

### Convolutional layer



## Phép tính convolution

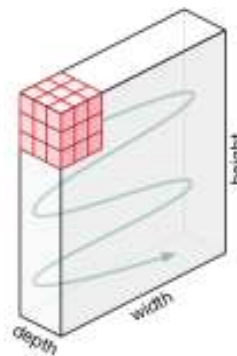
www.tvhungdhn@gmail

### Convolutional layer

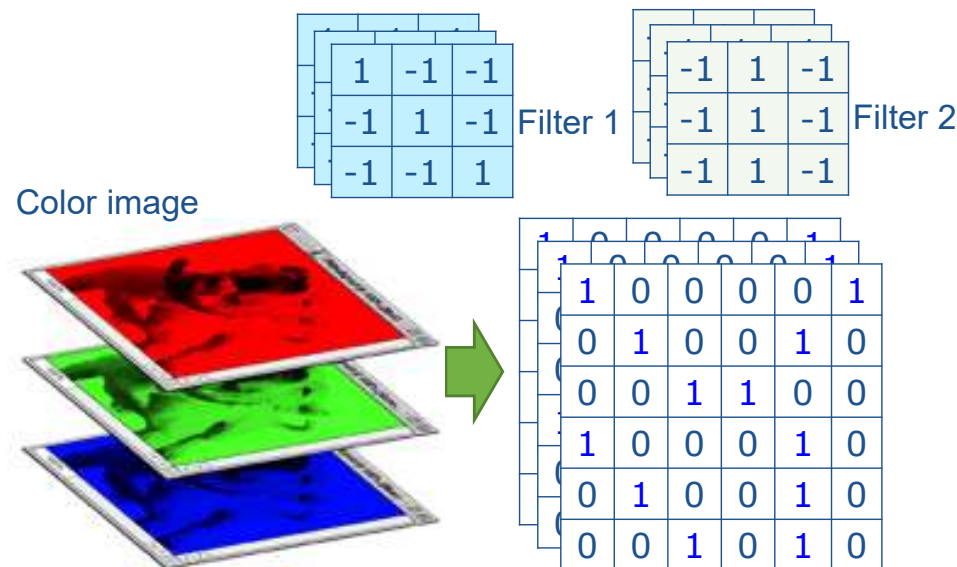
#### Ảnh màu

❖ Với ảnh màu có tới 3 kênh red, green, blue nên khi biểu diễn ảnh dưới dạng tensor 3 chiều. Ta cũng định nghĩa kernel là 1 tensor 3 chiều kích thước  $k \times k \times 3$ .

❖ Ta định nghĩa kernel có cùng độ sâu (depth) với biểu diễn ảnh, sau đó thực hiện di chuyển khối kernel tương tự như khi thực hiện trên ảnh xám.



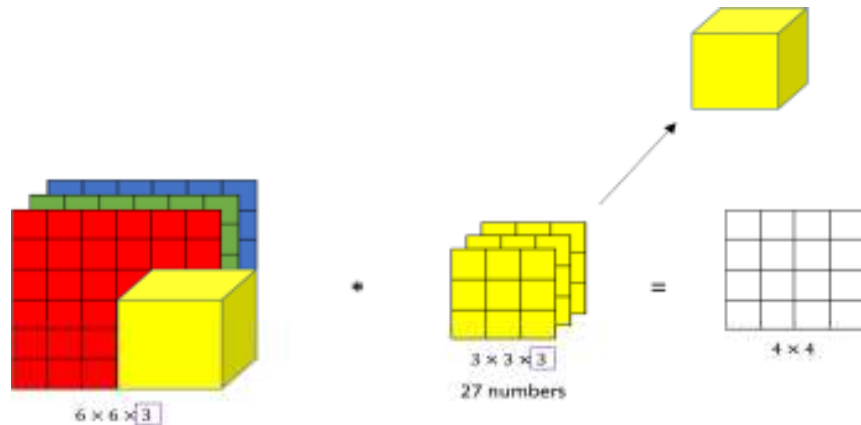
## Phép tính convolution cho ảnh màu



## Convolution cho ảnh màu

### Hoạt động tích chập trên hình ảnh RGB

- Ảnh RGB được biểu diễn bằng tensor 3D:  $(x, y, 3)$ , trong đó  $x, y$  là kích thước pixel và 3 tương ứng với 3 kênh màu. Bộ lọc cũng sẽ có 3 kênh.



## Convolution cho ảnh màu

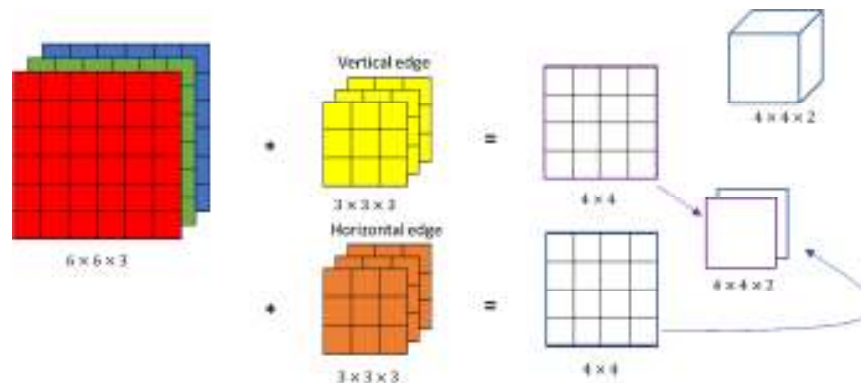
### Hoạt động tích chập trên hình ảnh RGB

Kích thước kênh trong kernel cho phép phát hiện các đặc trưng trong một tập con của kích thước ảnh, ví dụ:

- Các cạnh theo chiều dọc chỉ có kích thước màu đỏ
  - Cho tất cả các ô trong kênh màu xanh lam và xanh lục về 0
- Các cạnh theo chiều ngang có kích thước màu đỏ và màu xanh
- Các cạnh theo chiều dọc bất kể kích thước màu sắc

## Convolution cho ảnh màu

Chúng ta có thể xác định các đặc trưng khác nhau cùng một lúc bằng cách sử dụng nhiều bộ lọc



Đầu ra sẽ có số kênh bằng với số đặc trưng đang tìm để phát hiện

Dit document is beschikbaar op

**studeersnel**

Downloaded by Quan Dang (dangquan3172006@gmail.com)

## Convolution cho ảnh màu

www.tvhungdhcn@gmail

### Convolutional layer

### Ảnh màu

Khi biểu diễn ma trận ta cần 2 chỉ số hàng và cột:  $i$  và  $j$ , thì khi biểu diễn ở dạng tensor 3 chiều cần thêm chỉ số độ sâu  $k$ . Nên chỉ số mỗi phần tử trong tensor là  $x_{ijk}$

|   |     |     |     |     |   |
|---|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0 |
| 0 | 156 | 158 | 156 | 158 | 0 |
| 0 | 153 | 154 | 157 | 159 | 0 |
| 0 | 149 | 151 | 155 | 159 | 0 |
| 0 | 146 | 146 | 149 | 153 | 0 |
| 0 | 0   | 0   | 0   | 0   | 0 |

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 0 | 1 | 1 |
| 0 | 1 | 1 |

|   |     |     |     |     |   |
|---|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0 |
| 0 | 167 | 166 | 167 | 158 | 0 |
| 0 | 164 | 165 | 168 | 159 | 0 |
| 0 | 160 | 162 | 166 | 159 | 0 |
| 0 | 146 | 146 | 149 | 153 | 0 |
| 0 | 0   | 0   | 0   | 0   | 0 |

|   |   |   |
|---|---|---|
| 1 | 0 | 0 |
| 1 | 1 | 1 |
| 1 | 0 | 1 |

|   |     |     |     |     |   |
|---|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0 |
| 0 | 163 | 162 | 163 | 158 | 0 |
| 0 | 160 | 161 | 164 | 159 | 0 |
| 0 | 156 | 158 | 162 | 159 | 0 |
| 0 | 146 | 146 | 149 | 153 | 0 |
| 0 | 0   | 0   | 0   | 0   | 0 |

|   |   |   |
|---|---|---|
| 0 | 1 | 1 |
| 0 | 1 | 0 |
| 1 | 1 | 1 |

|  |  |  |  |  |  |
|--|--|--|--|--|--|
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |

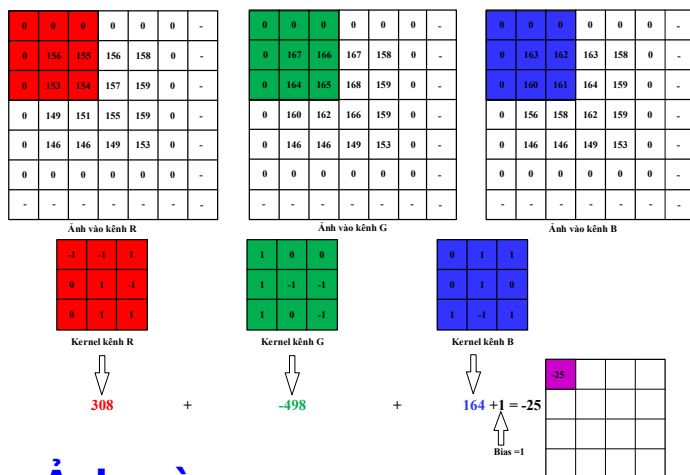


# Convolution cho ảnh màu

www.tvhungdhcn@gmail

## Convolutional layer

$$y_{11} = b + (x_{111} * w_{111} + x_{121} * w_{121} + x_{131} * w_{131} + x_{211} * w_{211} + x_{221} * w_{221} + x_{231} * w_{231} + x_{311} * w_{311} + x_{321} * w_{321} + x_{331} * w_{331}) + (x_{112} * w_{112} + x_{122} * w_{122} + x_{132} * w_{132} + x_{212} * w_{212} + x_{222} * w_{222} + x_{232} * w_{232} + x_{312} * w_{312} + x_{322} * w_{322} + x_{332} * w_{332}) + (x_{113} * w_{113} + x_{123} * w_{123} + x_{133} * w_{133} + x_{213} * w_{213} + x_{223} * w_{223} + x_{233} * w_{233} + x_{313} * w_{313} + x_{323} * w_{323} + x_{333} * w_{333}) = -25$$



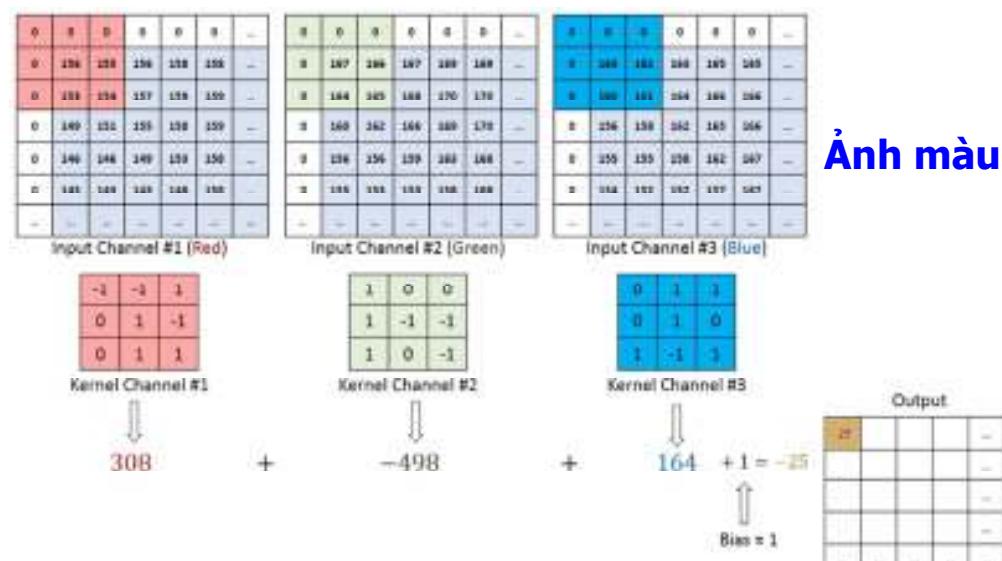
Ảnh màu



# Convolution cho ảnh màu

www.tvhungdhcn@gmail

## Convolutional layer



Ảnh màu

# Ý nghĩa của phép tính convolution

www.tvhungdhcn@gmail

- Mục đích của phép tính convolution trên ảnh là làm mờ, làm nét ảnh; xác định các đường;... Mỗi kernel khác nhau thì sẽ phép tính convolution sẽ có ý nghĩa khác nhau. Ví dụ:

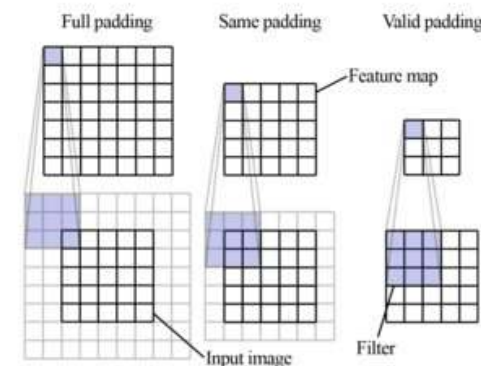
| Operation             | Kernel w                                                                        | Image result g(x,y) |
|-----------------------|---------------------------------------------------------------------------------|---------------------|
| Identity              | $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$             |                     |
| Edge detection        | $\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$           |                     |
|                       | $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$            |                     |
|                       | $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$     |                     |
| Sharpen               | $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$         |                     |
| Box blur (normalized) | $\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ |                     |

Downloaded by Quan Dang (dangquan3172006@gmail.com)

# Padding (đệm) và Stride (sải bước)

Thông thường, tích chập làm giảm ngỏ ra hay mất thông tin ở các góc của pixel. Để giải quyết điều này ta có các tùy chọn sau:

- Full-padding:** cho kích thước kernel là k, p = k-1 trong đó p là số hàng hoặc cột 0 được thêm vào
- Same-padding:** cho kích thước kernel là k, p =  $\lceil \frac{k}{2} \rceil$  trong đó p là số hàng hoặc cột 0 được thêm vào
- Valid-padding:** Kernel tích chập chỉ được phép truy cập vào các vị trí mà toàn bộ kernel chứa tất cả các ngỏ vào



## Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Như ở trên mỗi lần thực hiện phép tính convolution xong thì kích thước ma trận Y đều nhỏ hơn X. Tuy nhiên ta muốn ma trận Y thu được có kích thước bằng ma trận X → Tìm cách giải quyết cho các phần tử ở viền → Thêm giá trị 0 ở viền ngoài ma trận X.

Ma trận X  
khi thêm  
viền 0 bên  
ngoài

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |



## Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Như vậy ta đã giải quyết được vấn đề tìm A cho phần tử  $x_{1,1}$  và ma trận Y thu được sẽ bằng kích thước ma trận X ban đầu.
- Phép tính này gọi là convolution với padding=1. Padding=k nghĩa là thêm k vector 0 vào mỗi phía của ma trận.



## Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Cũng như ở trên ta thực hiện tuần tự các phần tử trong ma trận X, thu được ma trận Y cùng kích thước ma trận X, ta gọi là **stride=1**.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

stride=1, padding=1

Dit document is beschikbaar op

Downloaded by Quan Dang (dangquan3172006@gmail.com)



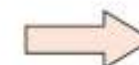
## Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Stride là số pixel thay đổi trên ma trận đầu vào. Khi stride là 1 thì ta di chuyển các kernel 1 pixel. Khi stride là 2 thì ta di chuyển các kernel đi 2 pixel và tiếp tục như vậy. Hình dưới là lớp tích chập hoạt động với stride là 2.

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 11 | 12 | 13 | 14 | 15 | 16 | 17 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| 31 | 32 | 33 | 34 | 35 | 36 | 37 |
| 41 | 42 | 43 | 44 | 45 | 46 | 47 |
| 51 | 52 | 53 | 54 | 55 | 56 | 57 |
| 61 | 62 | 63 | 64 | 65 | 66 | 67 |
| 71 | 72 | 73 | 74 | 75 | 76 | 77 |

Convolve with 3x3  
filters filled with ones



|     |     |  |
|-----|-----|--|
| 108 | 126 |  |
| 288 | 306 |  |
|     |     |  |



# Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Tuy nhiên nếu **stride=k** ( $k > 1$ ) thì ta chỉ thực hiện phép tính convolution trên các phần tử  $x_1 + i * k, 1 + j * k$ . Ví dụ  $k=2$

Hiểu đơn giản là bắt đầu từ vị trí  $x_{1,1}$  sau đó nhảy  $k$  bước theo chiều dọc và ngang cho đến hết ma trận X.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

stride=2,  
padding=1



# Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Ví dụ tích chập và tương quan chéo sử dụng hàm kernel 3x3 ảnh vào 5x5 đệm biên 1x1 giá trị 0, sử dụng bước sải 2x2.

|   |   |   |
|---|---|---|
| K |   |   |
| 2 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |



# Padding (đệm) và Stride (sải bước)

www.tvhungdhcn@gmail

- Kích thước của ma trận Y là 3\*3 đã giảm đi đáng kể so với ma trận X.
- Công thức tổng quát cho phép tính convolution của ma trận X kích thước  $m*n$  với kernel kích thước  $k*k$ ,  $stride = s$ ,  $padding = p$  ra ma trận Y kích thước.

$$\left(\frac{m - k + 2p}{s} + 1\right) * \left(\frac{n - k + 2p}{s} + 1\right)$$

- Stride thường dùng để giảm kích thước của ma trận sau phép tính convolution.



# Minh họa các kiểu padding và stride

www.tvhungdhcn@gmail

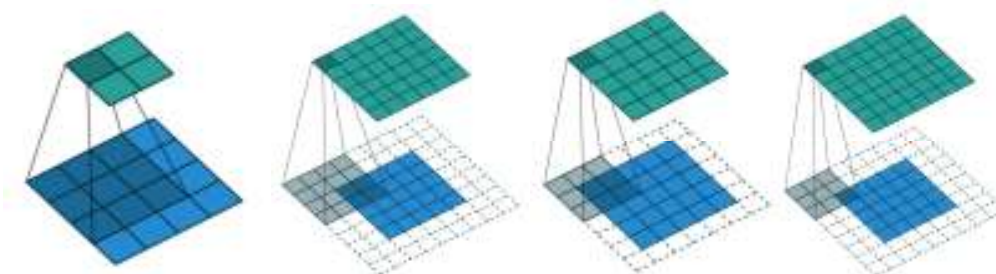
Màu xanh lam là đầu vào và màu lục lam là đầu ra.

Arbitrary padding, no strides

Padding =full, no strides

Half padding, no strides

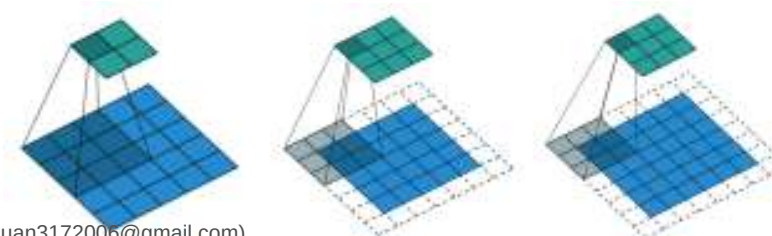
No padding, no strides



No padding, stride

Padding, strides

Padding, strides (odd)

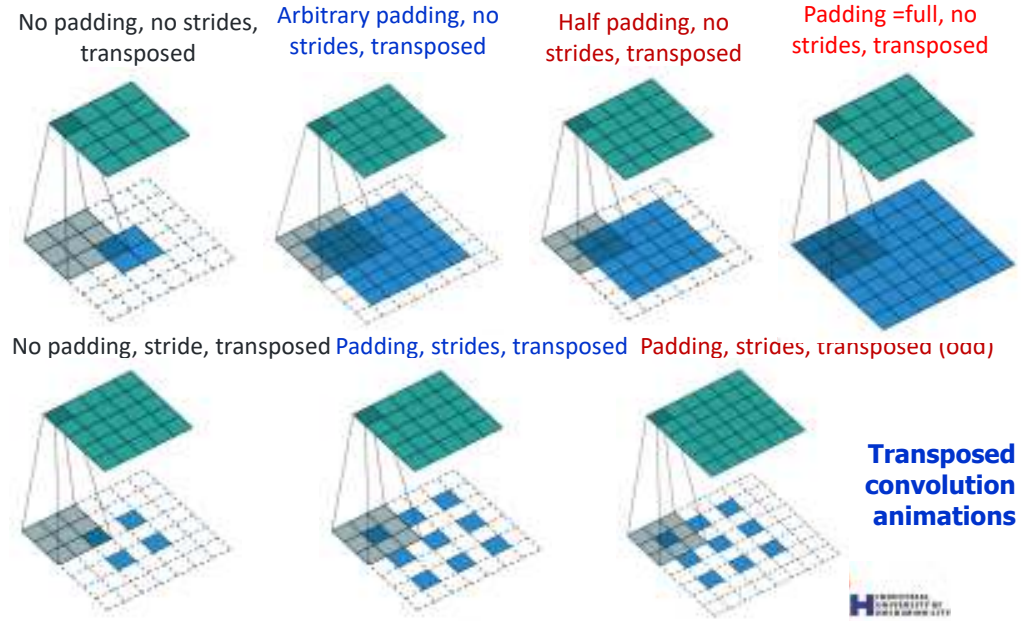


Convolution  
animations



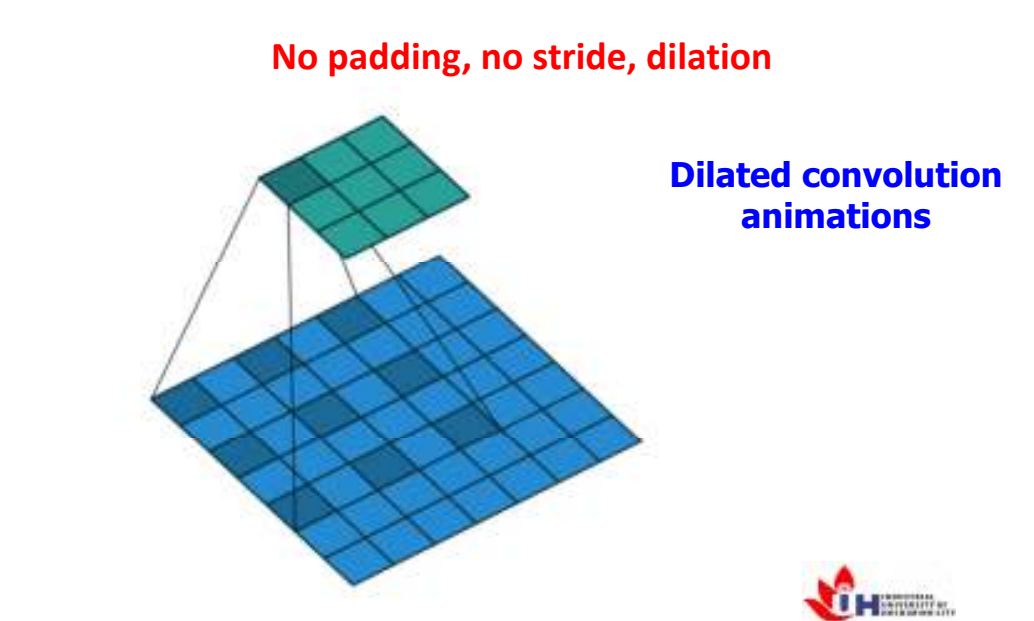
# Minh họa các kiểu padding và stride

www.tvhungdhcn@gmail.com      Màu xanh lam là đầu vào và màu lục lam là đầu ra.



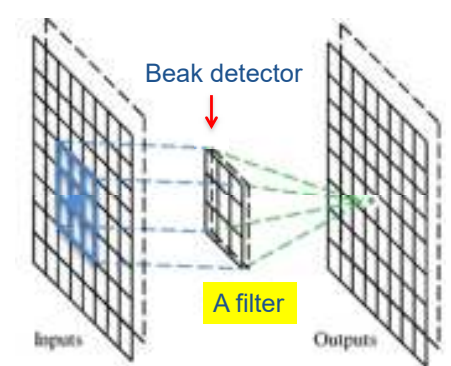
# Minh họa các kiểu padding và stride

www.tvhungdhcn@gmail.com      Màu xanh lam là đầu vào và màu lục lam là đầu ra.



# Mạng CNN (Convolutional neural network)

CNN là một mạng neural gồm nhiều lớp tích chập. Mỗi lớp chập có bộ lọc để thực hiện hoạt động tích chập.



Một lớp tích chập

# Mạng CNN (Convolutional neural network)

Convolutional Neural Networks (CNNs) là các mạng thần kinh sử dụng *Convolution* thay cho phép nhân ma trận chung trong ít nhất một trong các lớp của chúng

- Chúng thường được sử dụng để xử lý dữ liệu có cấu trúc liên kết dạng ô lưới
  - Dữ liệu ảnh → mỗi ảnh có thể được coi là một ô lưới của pixel 2D
  - Dữ liệu chuỗi thời gian → mỗi chuỗi thời gian có thể được coi là ô lưới 1D lấy mẫu trong cùng không gian thời gian

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Convolutional Neural Network là gì

Convolutional Neural Network (CNNs – Mạng neural tích chập) là một trong những mô hình Deep Learning tiên tiến. Nó giúp cho chúng ta xây dựng được những hệ thống thông minh với độ chính xác cao như hiện nay.

CNN được sử dụng nhiều trong các bài toán nhận dạng các object trong ảnh. Để tìm hiểu tại sao thuật toán này được sử dụng rộng rãi cho việc nhận dạng (detection), chúng ta hãy tìm hiểu về thuật toán này.



# Mạng CNN (Convolutional neural network)

## Convolution là gì?

❖ Tổng trọng số di chuyển

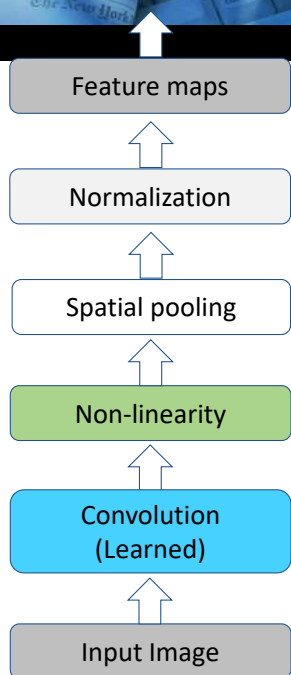


Input

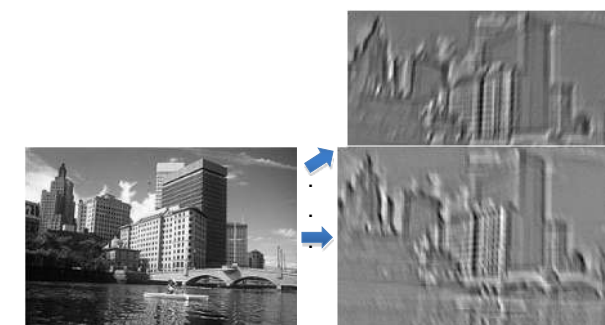
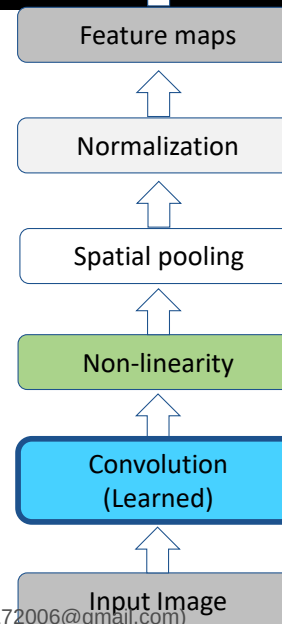


Feature Activation Map

# Mạng CNN (Convolutional neural network)



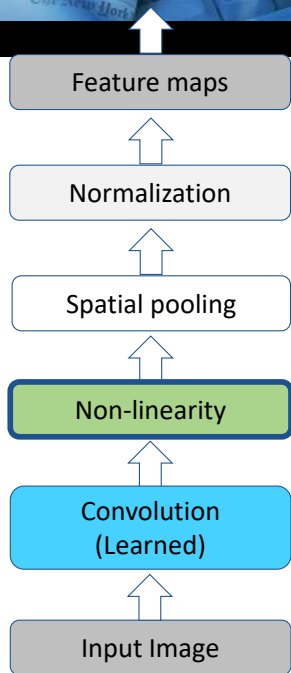
# Mạng CNN (Convolutional neural network)



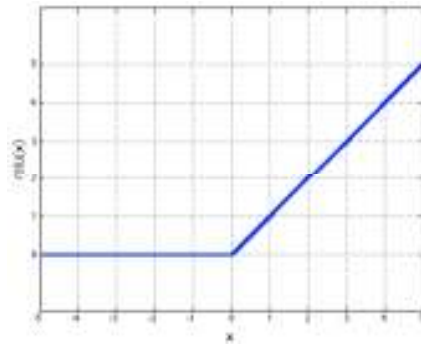
Input

Feature Map

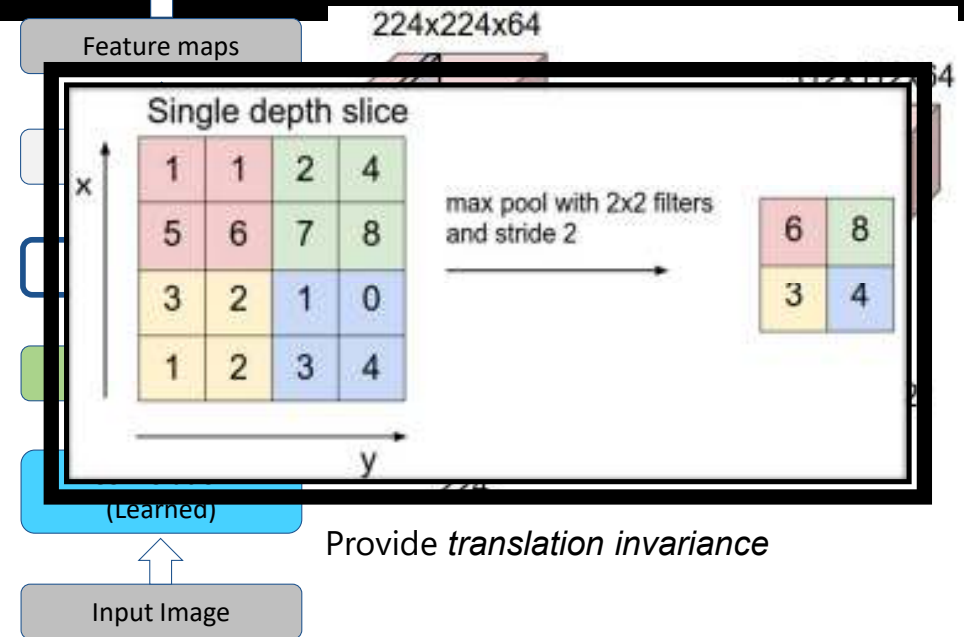
# Mạng CNN (Convolutional neural network)



Rectified Linear Unit (ReLU)



# Mạng CNN (Convolutional neural network)



# Mạng CNN (Convolutional neural network)



Big labeled datasets

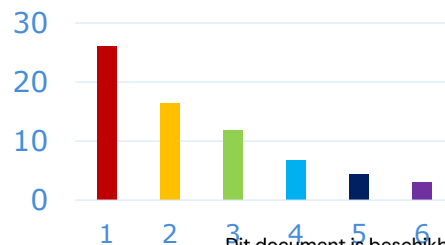
Deep learning



GPU technology

Gần đây lĩnh vực này tác động đáng kể

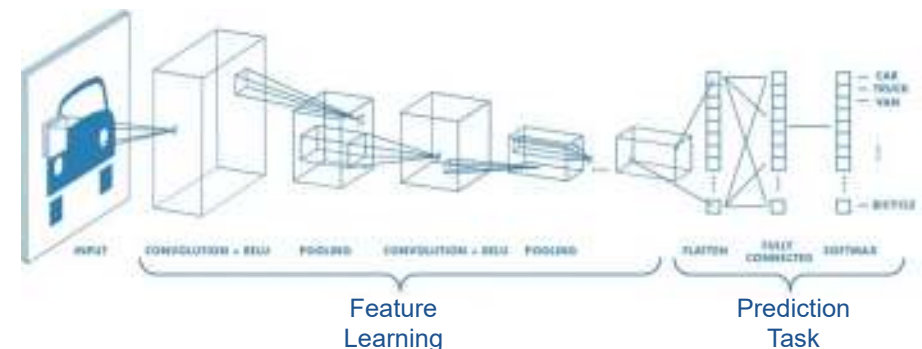
ImageNet top-5 error (%)



Dit document is beschikbaar op

# Cấu trúc mô hình CNN

Thông thường, mô hình CNN bao gồm các lớp tích chập, để tìm chọn đặc trưng, theo sau là các lớp fully connected để thực hiện nhiệm vụ tiên đoán.



## Cấu trúc mô hình CNN

Một lớp học đặc trưng tiêu biểu của mạng tích chập bao gồm ba giai đoạn:

- Giai đoạn đầu tiên thực hiện song song một số phép toán tích chập để tạo ra một tập hàm kích hoạt tuyến tính
  - Mỗi lần tích chập sử dụng một kernel khác nhau để tìm các đặc trưng khác nhau
  - Ngõ vào thường là một ô lưới của các giá trị vector quan sát
- Giai đoạn thứ hai, mỗi hàm kích hoạt tuyến tính được thực hiện thông qua hàm kích hoạt phi tuyến (ví dụ: ReLU)
- Giai đoạn thứ ba, hàm gộp được sử dụng để thay đổi thêm ngõ ra của lớp

## Mạng CNN (CNN, ConvNet, DCN)

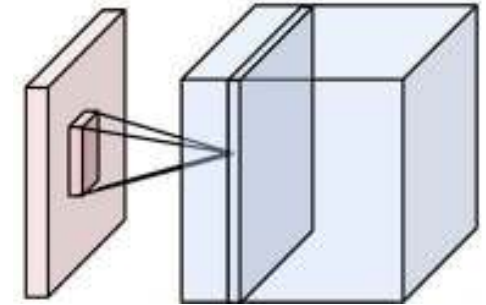
❖ **CNN = mạng neuron nhiều lớp với**

■ **Liên kết cục bộ (Local connectivity):**

- Các neuron trong một lớp chỉ được kết nối với một vùng nhỏ của lớp trước nó

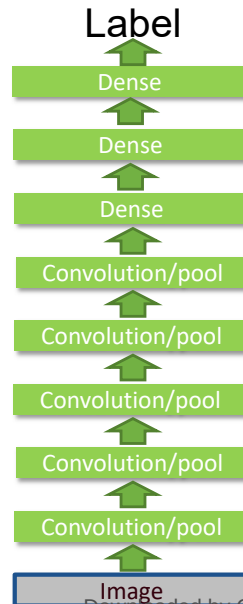
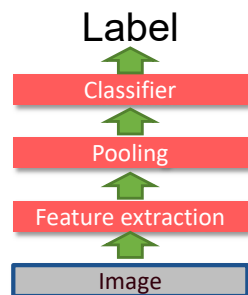
■ **Chia sẻ trọng số tham số trên các không gian vị trí (Share weight parameters across spatial) positions:**

- Học các bộ lọc kernel dịch bất biến



## Các đặc trưng được thiết kế và học

Các bộ lọc tích chập được huấn luyện có giám sát bằng phân loại lỗi lan truyền ngược



## Ứng dụng: ImageNet

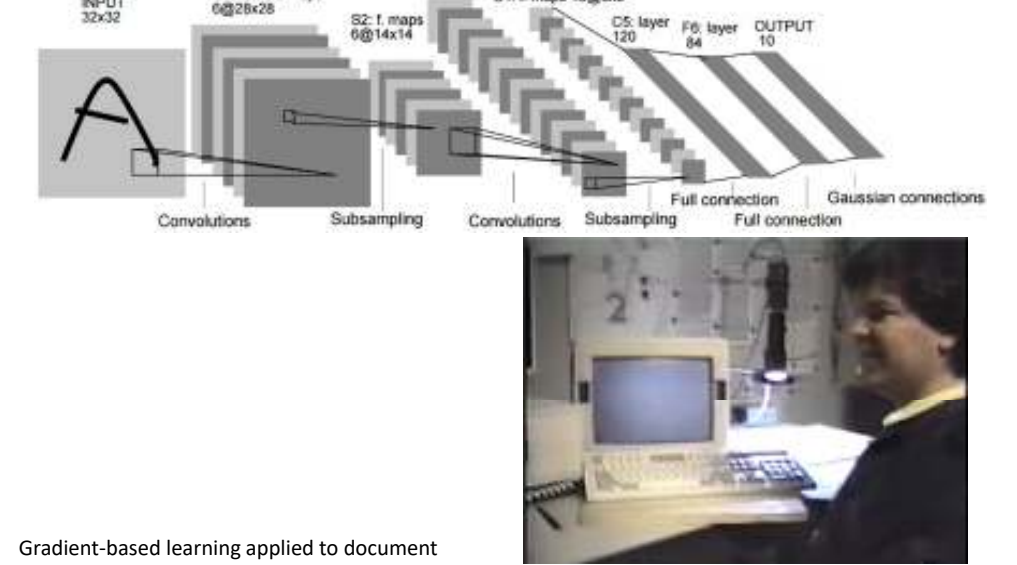
IMAGENET



[Deng et al. CVPR 2009]

- ~14 triệu hình ảnh được gắn nhãn, 20 nghìn lớp
- Nhãn con người thông qua Amazon Turk
- Hình ảnh được sưu tầm từ Internet

LeNet [LeCun et al. 1998]

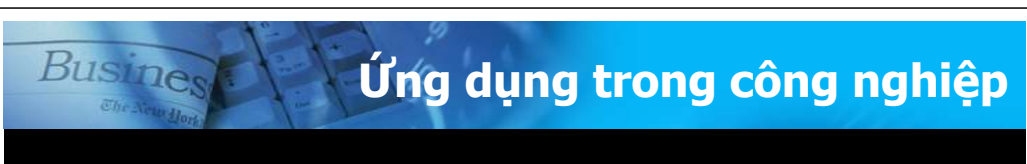


A photograph showing a person from the side, looking at a computer monitor. A microscope is mounted on top of the monitor. The person is wearing a dark shirt with a yellow collar. The background is a wall with some papers and a large number '2'.

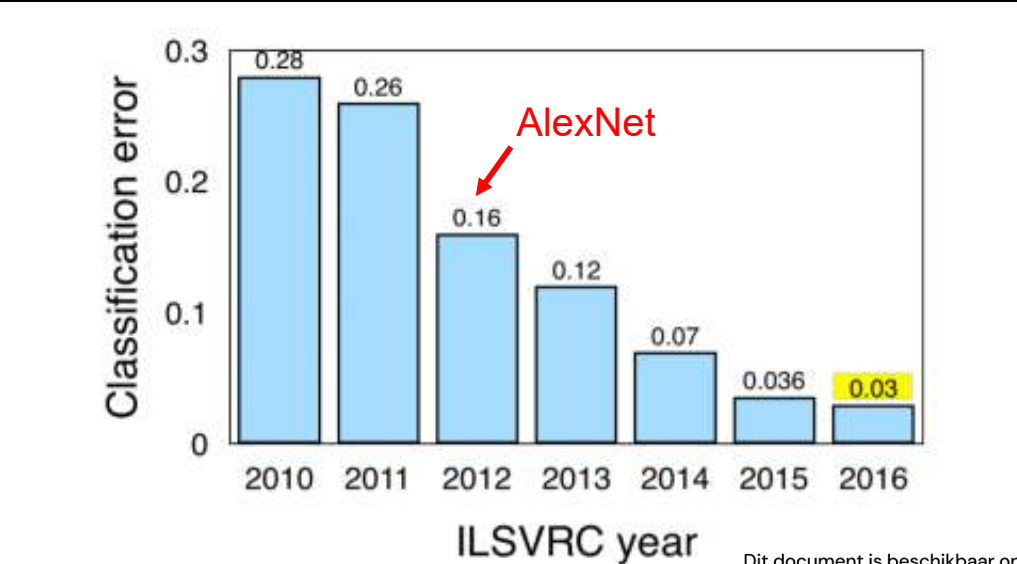
recognition [[LeCun, Bottou, Bengio, Haffner 1998](#)]

**AlexNet**

- Framework tương tự như LeCun'98 nhưng:
    - Mô hình lớn hơn (7 lớp ẩn, 650.000 đơn vị, 60.000.000 thông số)
    - Thêm dữ liệu ( $10^6$  so với  $10^3$  hình ảnh)
    - Triển khai GPU (tăng tốc 50 lần so với CPU)
    - Được huấn luyện cho hai GPU trong một tuần
- 
- The diagram illustrates a deep convolutional neural network (CNN) architecture. It begins with an input layer of size 224x224x3. This is followed by three convolutional layers, each with a max pooling operation. The first convolutional layer has a kernel size of 5x5 and a stride of 4, resulting in a 55x55x48 output. The second convolutional layer has a kernel size of 5x5 and a stride of 3, resulting in a 27x27x128 output. The third convolutional layer has a kernel size of 5x5 and a stride of 3, resulting in a 13x13x192 output. These are followed by three fully connected layers (dense layers) with sizes 2048, 2048, and 1000. The final output is a 1000-unit vector.



**Thách thức phân loại ImageNet**



**Ứng dụng trong công nghiệp**

- Được sử dụng trong Facebook, Google, Microsoft
  - Nhận dạng hình ảnh, Nhận dạng giọng nói, ....
  - Nhanh chóng trong thời gian thực nghiệm
- 
- conv2d, flatten, conv2d, maxpooling, ReLU, Softmax & Localization
- Representation @ 15x15x3x3
- C1: 3x3x3x3 @ 14x14
- M2: 3x3x3x3 @ 7x7
- C3: 1x3x3x3 @ 3x3
- I4: 5x5x3x3 @ 3x3
- I5: 3x3x3x3 @ 3x3
- I6: 1x1x3x3 @ 2x2
- Representation
- 5x5 labels

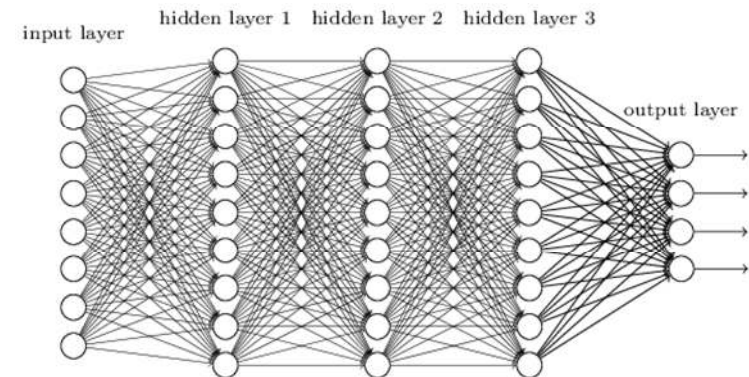


## Tóm lại

- ❖ Mạng neural, mạng perceptron nhiều lớp
  - Xem mạng neural như hệ thống phân cấp **Học** các đặc trưng
- ❖ Mạng CNN
  - Kiến trúc mạng cho cấu trúc hình ảnh
  - Nhận dạng các pixel từ “End-to-end”
  - Cùng với dữ liệu lớn (được gán nhãn) và nhiều tính toán → thành công lớn về điểm chuẩn, phân loại hình ảnh và hơn thế

## Mạng nhỏ hơn: CNN

- ❖ Việc học từ một mô hình nhỏ là rất tốt.
- ❖ Từ mô hình fully connected này, chúng ta có thực sự cần tất cả các góc cạnh không?
- ❖ Một số trong số này có thể được chia sẻ?



## Khảo sát quá trình **Học**

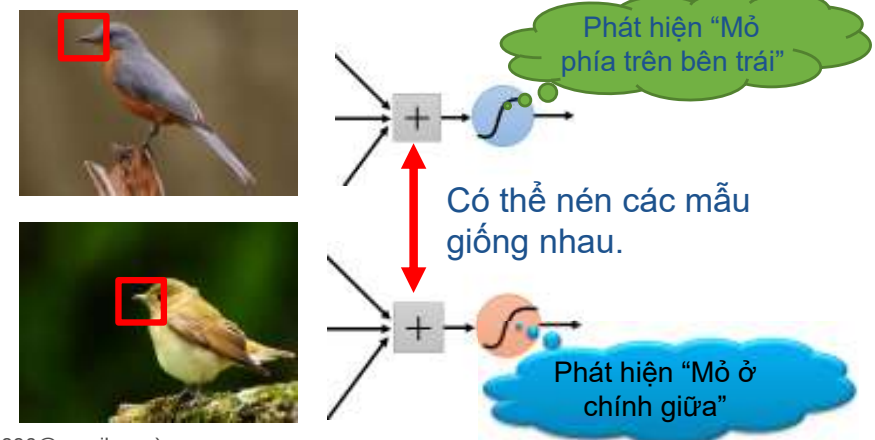
- ❖ Một vài mẫu nhỏ hơn nhiều so với toàn bộ ảnh

**Có thể đại diện cho một vùng nhỏ với 1 vài tham số**



## Khảo sát quá trình học

Mẫu giống nhau xuất hiện ở những nơi khác nhau: Chúng có thể được nén! Còn việc huấn luyện nhiều ngõ phát hiện “nhỏ” như vậy mà mỗi ngõ phát hiện đều phải “di chuyển”.

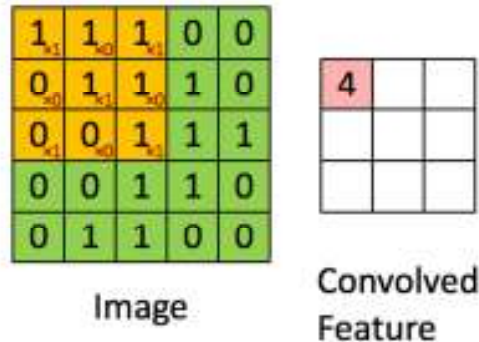


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Convolutional trong mạng CNN

Là một cửa sổ trượt (Sliding Windows) trên một ma trận như mô tả hình sau:



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Convolutional trong mạng CNN

Các convolutional layer có các parameter (kernel) đã được học để tự điều chỉnh lấy ra những thông tin chính xác nhất mà không cần chọn các feature.

Trong hình ảnh ví dụ trên, ma trận bên trái là một hình ảnh trắng đen được số hóa. Ma trận có kích thước  $5 \times 5$  và mỗi điểm ảnh có giá trị 1 hoặc 0 là giao điểm của dòng và cột.

Convolution hay tích chập là nhân từng phần tử trong ma trận 3. Sliding Window hay còn gọi là kernel, filter hoặc feature detect là một ma trận có kích thước nhỏ như trong ví dụ trên là  $3 \times 3$ .

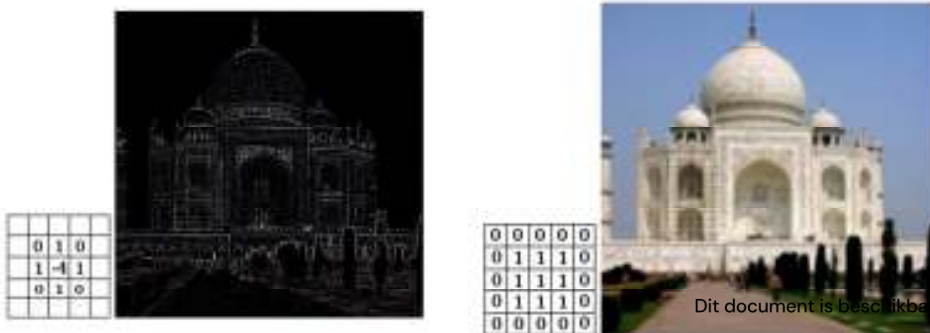


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Convolutional trong mạng CNN

Convolution hay tích chập là nhân từng phần tử bên trong ma trận  $3 \times 3$  với ma trận bên trái. Kết quả được một ma trận gọi là Convoled feature được sinh ra từ việc nhân ma trận Filter với ma trận ảnh  $5 \times 5$  bên trái.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Cấu trúc của mạng CNN

Mạng CNN là một tập hợp các lớp Convolution chồng lên nhau và sử dụng các hàm nonlinear activation như **ReLU**, **Softmax** và **tanh** để kích hoạt các trọng số trong các node.

Mỗi một lớp sau khi thông qua các hàm kích hoạt sẽ tạo ra các thông tin trừu tượng hơn cho các lớp tiếp theo. Trong mô hình mạng truyền ngược (feedforward neural network) thì mỗi neural đầu vào (input node) cho mỗi neural đầu ra trong các lớp tiếp theo.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Cấu trúc của mạng CNN

Mô hình này gọi là mạng kết nối đầy đủ (fully connected layer) hay mạng toàn vẹn (affine layer). Còn trong mô hình CNNs thì ngược lại. Các layer liên kết được với nhau thông qua cơ chế convolution.

Layer tiếp theo là kết quả convolution từ layer trước đó, nhờ vậy mà ta có được các kết nối cục bộ. Như vậy mỗi neural ở lớp kế tiếp sinh ra từ kết quả của filter áp đặt lên một vùng ảnh cục bộ của neural trước đó.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Hàm Relu:

- ReLU viết tắt của Rectified Linear Unit, là 1 hàm phi tuyến. Với đầu ra là:  $f(x) = \max(0, x)$ .
- Tại sao ReLU lại quan trọng: ReLU giới thiệu tính phi tuyến trong ConvNet. Vì dữ liệu trong thế giới mà chúng ta tìm hiểu là các giá trị tuyến tính không âm.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

Các mô hình hàm kích hoạt của một mạng neural

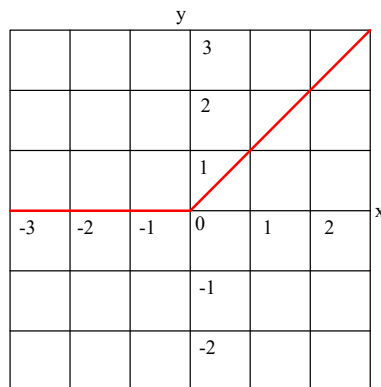
## Hàm Relu:

- Biểu diễn hàm:

$$f(x) = \begin{cases} 0 & \text{khi } x < 0 \\ x & \text{khi } x \geq 0 \end{cases}$$

- Đạo hàm của hàm:

$$f'(x) = \begin{cases} 0 & \text{khi } x < 0 \\ 1 & \text{khi } x \geq 0 \end{cases}$$



Đồ thị hàm ReLU



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

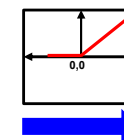
Các mô hình hàm kích hoạt của một mạng neural

## Hàm Relu:

Ví dụ

Hàm biến đổi

|     |      |     |     |
|-----|------|-----|-----|
| 15  | 20   | -10 | 35  |
| 18  | -110 | 25  | 100 |
| 20  | -15  | 25  | -10 |
| 101 | 75   | 18  | 23  |



|     |    |    |     |
|-----|----|----|-----|
| 15  | 20 | 0  | 35  |
| 18  | 0  | 25 | 100 |
| 20  | 0  | 25 | 0   |
| 101 | 75 | 18 | 23  |

Lớp ReLU



# Mạng CNN (Convolutional neural network)

www.tvhungdhn@gmail

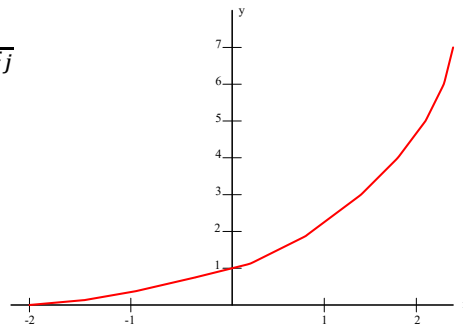
Các mô hình hàm kích hoạt của một mạng neural

## Hàm Softmax:

Công thức hàm softmax:  $s_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$

Trong đó:  $0 \leq s_i \leq 1$ , do  $e^x \geq 0$  với mọi  $i$ ,  $x.s$  là hàm đồng biến, do  $e^x$  là hàm đồng biến mà mẫu số lại không đổi nên  $s$  cũng là đồng biến. Tức là nếu  $x_i \leq x_j$  thì

$$s_i \leq s_j, \sum_j s_i = 1$$



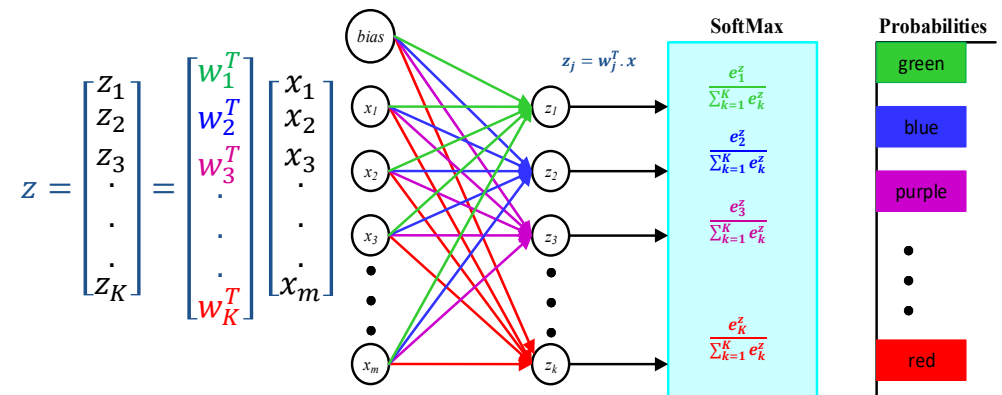
Hình Đồ thị hàm softmax.



# Mạng CNN (Convolutional neural network)

www.tvhungdhn@gmail

## Hàm Softmax:



Hàm softmax sử dụng trong mạng neural.



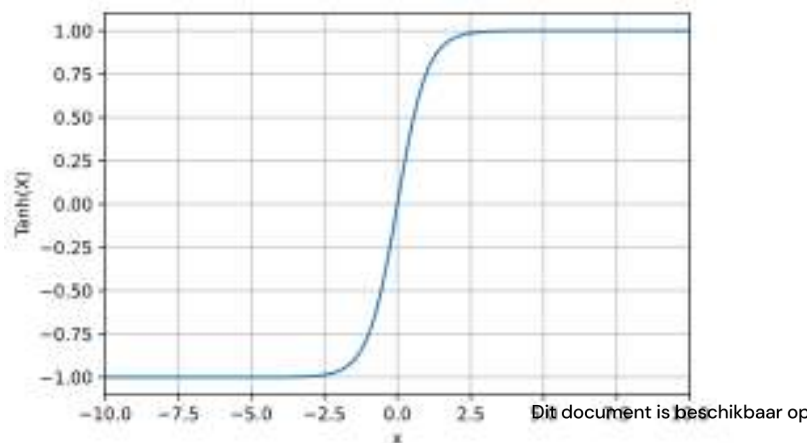
# Mạng CNN (Convolutional neural network)

www.tvhungdhn@gmail

## Hàm tanh:

Công thức:  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Phân tích:



Downloaded by: Quan Dang (dangquan3172006@gmail.com)



# Mạng CNN (Convolutional neural network)

www.tvhungdhn@gmail

## Hàm tanh:

Công thức:  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Phân tích:

Hàm tanh nhận đầu vào là một số thực và chuyển thành một giá trị trong khoảng  $(-1; 1)$ . Cũng như Sigmoid, hàm tanh bị bão hoà ở 2 đầu (gradient thay đổi rất ít ở 2 đầu). Tuy nhiên hàm tanh lại đối xứng qua 0 nên khắc phục được một nhược điểm của Sigmoid. Hàm tanh còn có thể được biểu diễn bằng hàm sigmoid như sau:  $\tanh(x) = 2\sigma(2x - 1)$



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Cấu trúc của mạng CNN

Mỗi một lớp được sử dụng các cửa sổ lọc khác nhau thông thường có hàng trăm hàng nghìn cửa sổ lọc như vậy và kết hợp kết quả của chúng lại. Ngoài ra có một số lớp khác như lớp pooling hay subsampling dùng để lọc lại các thông tin hữu ích (loại bỏ các thông tin nhiễu).

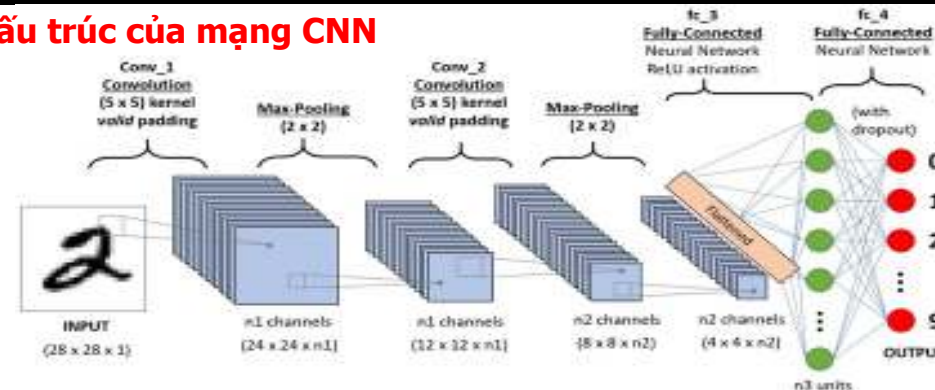
Trong quá trình huấn luyện mạng (training) CNN tự động học các giá trị qua các lớp lọc dựa vào cách thức thực hiện. Ví dụ trong việc phân lớp ảnh, CNNs sẽ tìm ra thông số tối ưu cho các filter tương ứng theo thứ tự: pixel  $\rightarrow$  edges  $\rightarrow$  shapes  $\rightarrow$  facial  $\rightarrow$  high-level features. Lớp cuối cùng được dùng để phân lớp ảnh.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Cấu trúc của mạng CNN



Trong mô hình CNN có 2 đặc tính cần quan tâm là tính bất biến (Location Invariance) và tính kết hợp (Compositionality). Với cùng một đối tượng, nếu đối tượng này được chiếu theo các góc độ khác nhau (translation, rotation, scaling) thì độ chính xác của thuật toán sẽ bị ảnh hưởng đáng kể.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Convolutional layer

### Nhận xét:

- ❖ Output Y của phép tính convolution trên ảnh màu là 1 matrix.
- ❖ Có 1 **hệ số bias** được cộng vào sau bước tính tổng các phần tử của phép tính element-wise
- ❖ Các quy tắc đối với **padding** và **stride** toàn hoàn tương tự như ở phần trước.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Convolutional layer

- ❖ Với mỗi kernel khác nhau sẽ học được những đặc trưng khác nhau của ảnh, nên trong mỗi convolutional layer sẽ dùng nhiều kernel để học được **nhiều đặc trưng** của ảnh. Vì mỗi kernel cho ra output là 1 matrix nên k kernel sẽ cho ra k output matrix. Việc kết hợp k output matrix này lại thành 1 tensor 3 chiều có chiều sâu k.

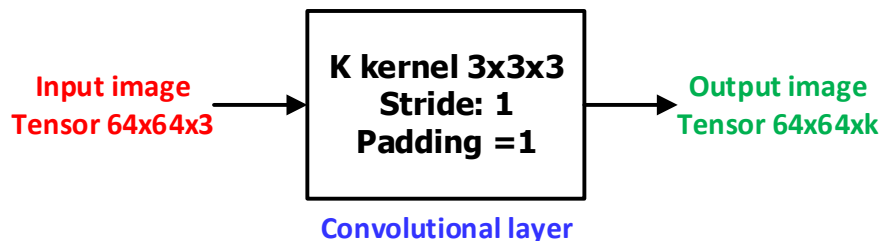


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Convolutional layer

Output của convolutional layer đầu tiên sẽ thành input của convolutional layer tiếp theo.



Convolutional layer đầu tiên

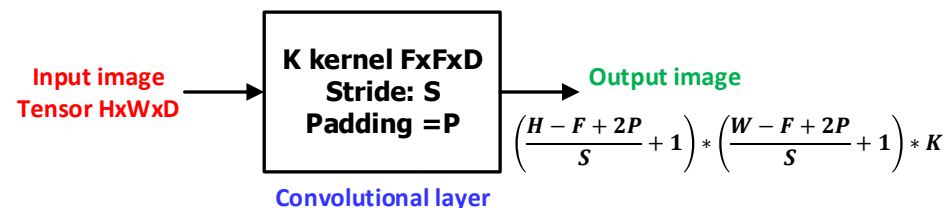


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Convolutional layer tổng quát

- ❖ Giả sử input của 1 convolutional layer tổng quát là tensor kích thước  $H * W * D$ .
- ❖ Kernel có kích thước  $F * F * D$  (kernel luôn có depth bằng depth của input và  $F$  là số lẻ), stride:  $S$ , padding:  $P$ .
- ❖ Convolutional layer áp dụng  $K$  kernel.
- ❖ Output của layer là tensor 3 chiều có kích thước:



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Convolutional layer tổng quát

Lưu ý:

- ❖ Output của convolutional layer sẽ qua hàm **activation function** trước khi trở thành input của convolutional layer tiếp theo.
- ❖ Tổng số các tham số của layer: Mỗi kernel có kích thước  $F * F * D$  và có 1 hệ số bias, nên tổng tham số của 1 kernel là  $F * F * D + 1$ . Mà convolutional layer áp dụng  $K$  kernel  $\Rightarrow$  Tổng số tham số trong layer này là  $K * (F * F * D + 1)$ .

Dit document is beschikbaar op



Downloaded by Quan Dang (dangquan3172006@gmail.com)

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Tóm lại lớp tích chập:

- Ảnh vào có kích thước  $W_1 \times H_1 \times D_1$
- Yêu cầu có 4 siêu tham số:
  - Kích thước bộ lọc  $K$
  - Phạm vi không gian  $F$
  - Bước rải  $S$
  - Số đệm bằng không  $P$
- Tạo ra ảnh có kích thước  $W_2 \times H_2 \times D_2$  với:
  - $W_2 = (W_1 - F + 2P) / S + 1$
  - $H_2 = (H_1 - F + 2P) / S + 1$  (tức là chiều rộng và cao được tính bằng nhau do đối xứng)
  - $D_2 = K$



Số đệm bằng không  $P$



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Tóm lại lớp tích chập:

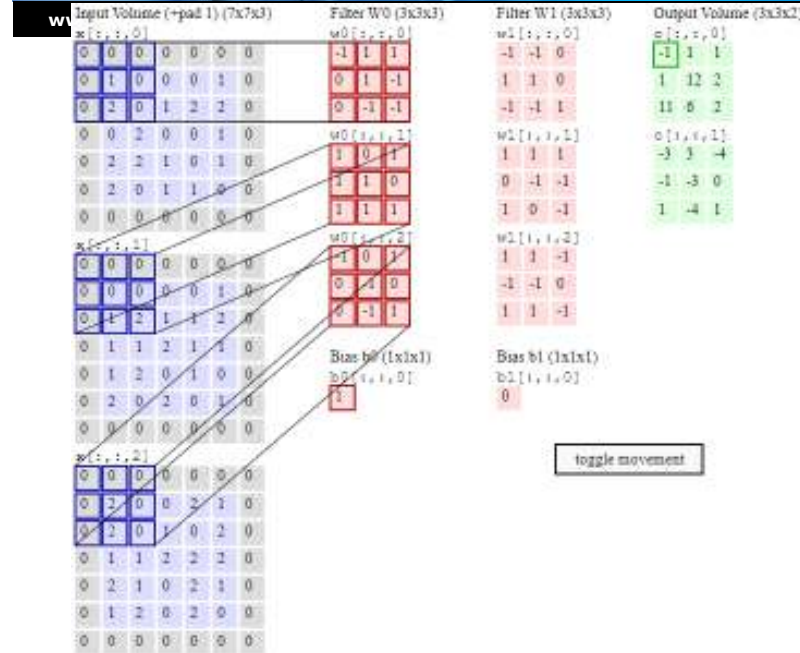
- Với tham số chia sẻ, trọng số cho mỗi bộ lọc F-F-D<sub>1</sub> bằng tổng trọng số (F-F-D<sub>1</sub>)·K với hệ số bias K.
- Với ảnh đầu ra độ sâu thứ d (có kích thước W<sub>2</sub>×H<sub>2</sub> là kết quả của việc thực hiện một phép tích hợp hợp lệ của bộ lọc thứ d trên ảnh đầu vào với bước sai là S và sau đó được bù lại bởi hệ số bias thứ d.

Nhìn chung các siêu tham số thường được chọn là F = 3, S = 1, P = 1. Tuy nhiên, cũng có những quy ước và quy tắc chung để lựa chọn phù hợp những siêu tham số này.



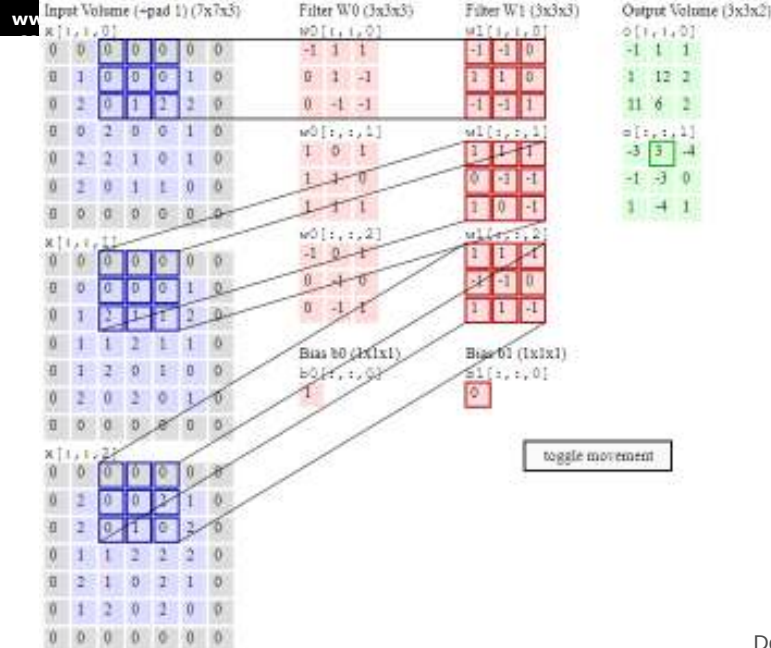
# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Pooling (tổng hợp) layer

Pooling layer sẽ cho ta tính bất biến đối với phép dịch chuyển (translation), phép quay (rotation) và phép co giãn (scaling). Tính kết hợp cục bộ cho ta các cấp độ biểu diễn thông tin từ mức độ thấp đến mức độ cao và trừu tượng hơn thông qua convolution từ các filter.

Đó là lý do tại sao CNNs cho ra mô hình với độ chính xác rất cao. Cũng giống như cách con người nhận biết các vật thể trong tự nhiên.

Mạng CNN sử dụng 3 ý tưởng cơ bản:

- Các trường tiếp nhận cục bộ (local receptive field)
- Trọng số chia sẻ (shared weights)
- Tổng hợp (pooling).



# Mạng CNN (Convolutional neural network)

## Tại sao phải Pooling?

Các pixel subsample sẽ không thay đổi đối tượng

bird



Subsampling



bird

Chúng ta có thể subsample các pixel để tạo hình ảnh nhỏ hơn



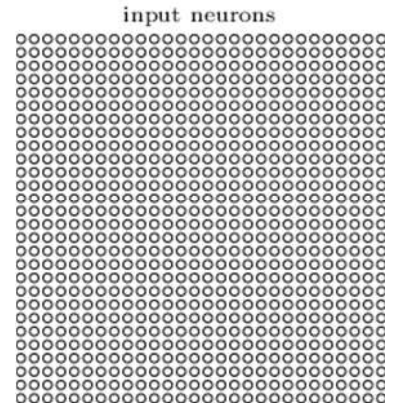
ít tham số hơn để mô tả hình ảnh

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Trường tiếp nhận cục bộ (local receptive field)

Đầu vào của mạng CNN là một ảnh. Ví dụ như ảnh có kích thước  $28 \times 28$  thì tương ứng đầu vào là một ma trận có  $28 \times 28$  và giá trị mỗi điểm ảnh là một ô trong ma trận. Trong mô hình mạng ANN (Artificial Neural Network) truyền thống thì chúng ta sẽ kết nối các neural đầu vào vào tầng ảnh.

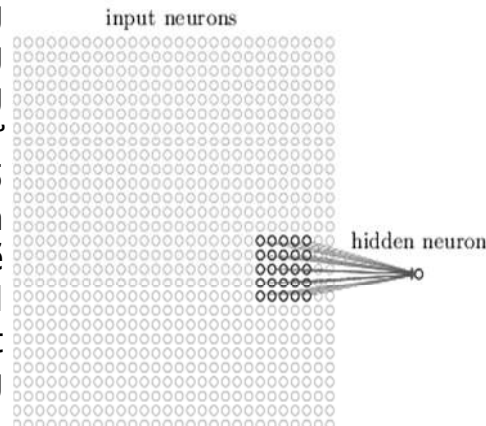


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Trường tiếp nhận cục bộ (local receptive field)

Tuy nhiên trong CNN chúng ta không làm như vậy mà chúng ta chỉ kết nối trong một vùng nhỏ của các neural đầu vào như một kernel có kích thước  $5 \times 5$  tương ứng  $(28 - 5 + 1) = 24$  điểm ảnh đầu vào. Mỗi một kết nối sẽ học một trọng số và mỗi neural ẩn sẽ học một bias. Mỗi một vùng  $5 \times 5$  đó gọi là một trường tiếp nhận cục bộ.



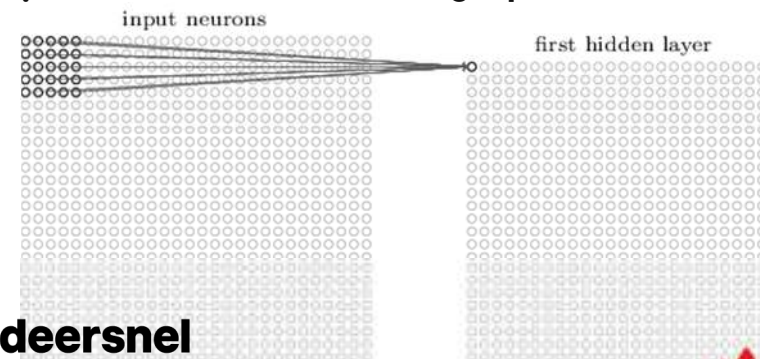
# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Trường tiếp nhận cục bộ (local receptive field)

Một cách tổng quan, ta có thể tóm tắt các bước tạo ra 1 hidden layer bằng các cách sau:

### 1. Tạo ra neural ẩn đầu tiên trong lớp ẩn 1

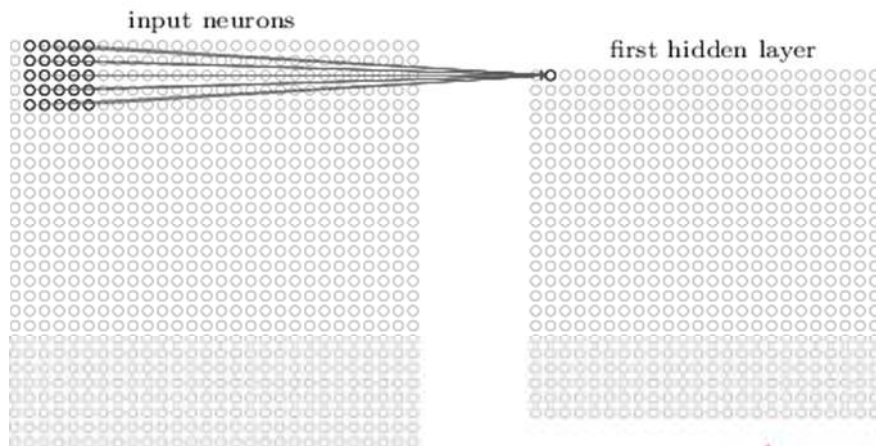


## Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

### Trường tiếp nhận cục bộ (local receptive field)

2. Dịch filter qua bên phải một cột sẽ tạo được neural ẩn thứ 2.



## Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

### Trường tiếp nhận cục bộ (local receptive field)

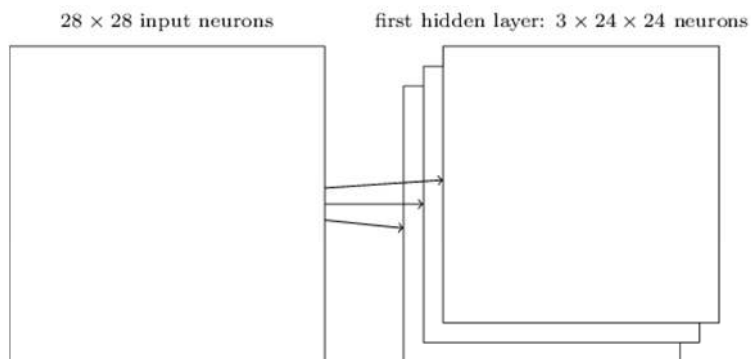
Với bài toán nhận dạng ảnh người ta thường gọi ma trận lớp đầu vào là feature map, trọng số xác định các đặc trưng là shared weight và độ lệch xác định một feature map là shared bias. Như vậy đơn giản nhất là qua các bước trên chúng ta chỉ có 1 feature map. Tuy nhiên trong nhận dạng ảnh chúng ta cần nhiều hơn một feature map.



## Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

### Trường tiếp nhận cục bộ (local receptive field)



Như vậy, local receptive field thích hợp cho việc phân tích dữ liệu ảnh, giúp chọn ra những vùng ảnh có giá trị tốt nhất cho việc đánh giá phân lớp.



Downloaded by Quan Dang (dangquan3172006@gmail.com)

## Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

### Trọng số chia sẻ (shared weight and bias)

Đầu tiên các trọng số cho mỗi filter (kernel) phải giống nhau. Tất cả các neural trong lớp ẩn đầu sẽ phát hiện chính xác feature tương tự chỉ ở các vị trí khác nhau trong hình ảnh đầu vào. Chúng ta gọi việc ánh xạ từ input layer sang hidden layer là một feature map. **Vậy mối quan hệ giữa số lượng feature map với số lượng tham số là gì?**



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Trọng số chia sẻ (shared weight and bias)

Chúng ta thấy mỗi feature map cần  $25 = 5 \times 5$  shared weight và 1 shared bias. Như vậy mỗi feature map cần  $5 \times 5 + 1 = 26$  tham số. Như vậy nếu có 10 feature map thì có  $10 \times 26 = 260$  tham số.

Còn với mô hình mạng ANN nếu layer đầu tiên có kết nối đầy đủ nghĩa là chúng ta có  $28 \times 28 = 784$  neural đầu vào như vậy ta chỉ có 30 neural ẩn. Như vậy ta cần  $28 \times 28 \times 30$  shared weight và 30 shared bias.



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Trọng số chia sẻ (shared weight and bias)

Tổng số tham số là  $28 \times 28 \times 30 + 30$  tham số lớn hơn nhiều so với CNN. Ví dụ vừa rồi chỉ mô tả để thấy được sự ước lượng số lượng tham số chứ chúng ta không so sánh được trực tiếp vì 2 mô hình khác nhau. Nhưng nếu mô hình có số lượng tham số ít hơn thì nó sẽ chạy nhanh hơn.

Tóm lại, một convolutional layer bao gồm các feature map khác nhau. Mỗi một feature map giúp phát hiện một vài đặc trưng trong bức ảnh. Lợi ích lớn nhất của trọng số chia sẻ là giảm tối đa số lượng tham số trong mạng CNN.

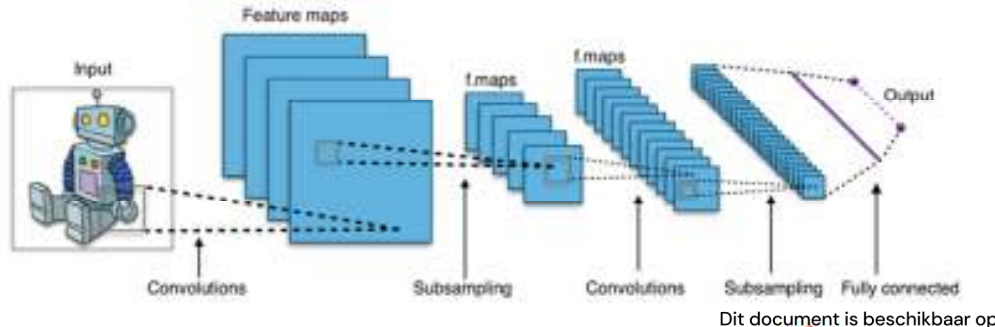


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Lớp tổng hợp (pooling layer)

Lớp pooling thường được sử dụng ngay sau lớp convolutional để đơn giản hóa thông tin đầu ra để giảm bớt số lượng neural.



Dit document is beschikbaar op



# Mạng CNN (Convolutional neural network)

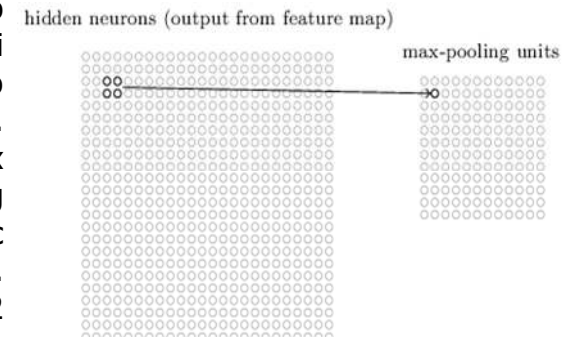
www.tvhungdhcn@gmail

## Lớp tổng hợp (pooling layer)

Phương pháp pooling phổ biến là max-pooling, phương pháp này chọn giá trị lớn nhất trong vùng đầu vào  $2 \times 2$ .

Như vậy qua lớp Max Pooling thì số lượng neural giảm đi 1 nửa.

Trong một mạng CNN có nhiều Feature Map nên mỗi Feature Map chúng ta sẽ cho mỗi Max Pooling khác nhau. Chúng ta có thể thấy rằng Max Pooling là cách hỏi xem trong các đặc trưng này thì đặc trưng nào là nổi trội nhất. Ngoài Max Pooling còn có L2



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Pooling (gộp) layer

- ❖ Pooling layer thường được dùng giữa các convolutional layer, để giảm kích thước dữ liệu nhưng vẫn giữ được các thuộc tính quan trọng. Kích thước dữ liệu giảm giúp giảm việc tính toán trong mô hình.
- ❖ Gọi pooling size có kích thước  $K \times K$ . Input của pooling layer có kích thước  $H \times W \times D$ , ta tách ra làm  $D$  ma trận kích thước  $H \times W$ . Với mỗi ma trận, trên vùng kích thước  $K \times K$  trên ma trận ta tìm maximum hoặc average của dữ liệu rồi viết vào ma trận kết quả. Quy tắc về stride và padding áp dụng như phép tính convolution trên ảnh.

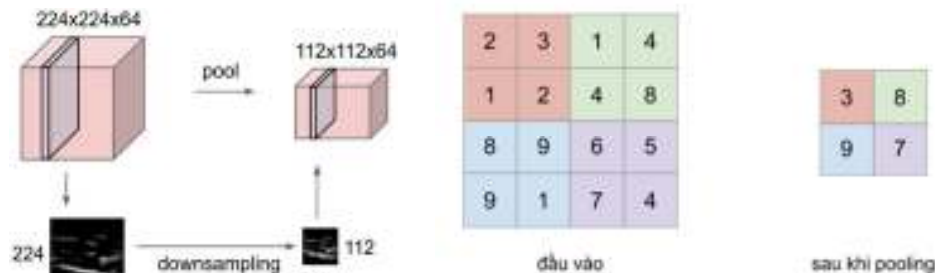


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Pooling layer

Nhưng hầu hết khi dùng pooling layer thì sẽ dùng  $\text{size}=(2,2)$ ,  $\text{stride}=2$ ,  $\text{padding}=0$ . Khi đó output width và height của dữ liệu giảm đi một nửa, depth thì được giữ nguyên.



Sau pooling layer ( $2 \times 2$ ).



# Mạng CNN (Convolutional neural network)

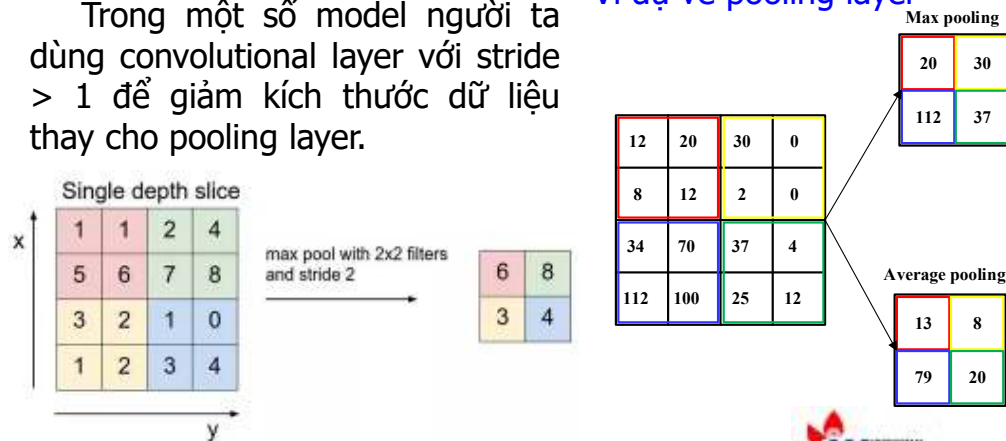
www.tvhungdhcn@gmail

## Mạng neural tích chập: Pooling layer

Có 2 loại pooling layer phổ biến là: max pooling và average pooling.

Trong một số model người ta dùng convolutional layer với  $\text{stride} > 1$  để giảm kích thước dữ liệu thay cho pooling layer.

Ví dụ về pooling layer



# Mạng CNN (Convolutional neural network)

Max Pooling

|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |

Filter 1

|    |   |    |
|----|---|----|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

Filter 2

|    |    |    |    |
|----|----|----|----|
| 3  | -1 | -3 | -1 |
| -3 | 1  | 0  | -3 |
| -3 | -3 | 0  | 1  |
| 3  | -2 | -2 | -1 |

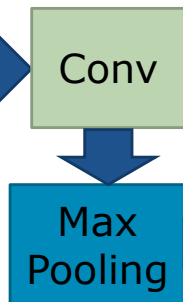
|    |    |    |    |
|----|----|----|----|
| -1 | -1 | -1 | -1 |
| -1 | -1 | -2 | 1  |
| -1 | -1 | -2 | 1  |
| -1 | 0  | -4 | 3  |

# Mạng CNN (Convolutional neural network)

## Max Pooling

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

6 x 6 image



Ảnh mới sẽ nhỏ hơn

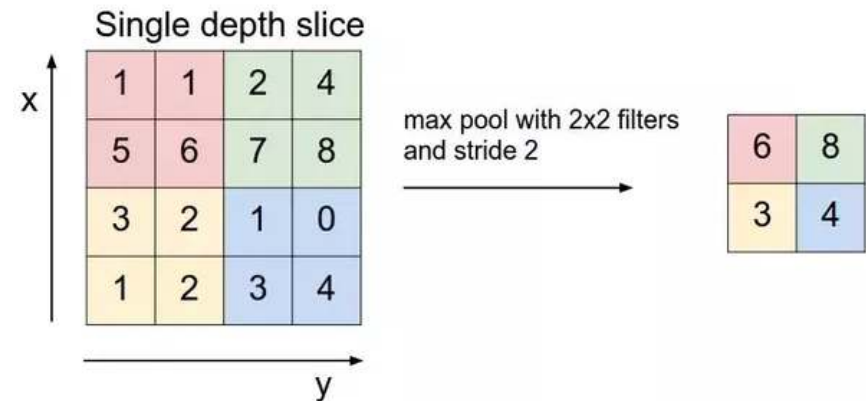


2 x 2 image

Mỗi bộ lọc cho 1 kênh

# Mạng CNN (Convolutional neural network)

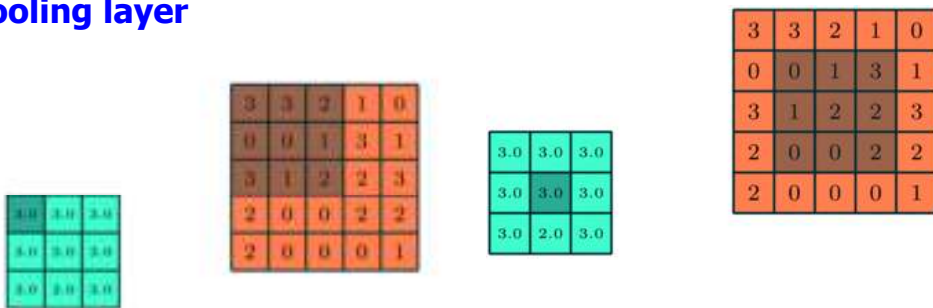
## Ví dụ max pooling



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Pooling layer



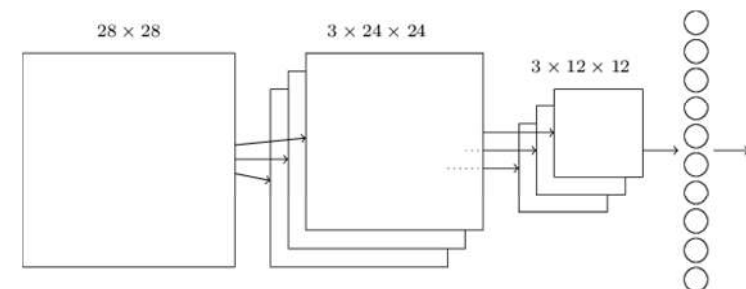
max pooling layer với size=(3,3), stride=1, padding=0

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Fully connected layer

Cuối cùng ta đặt tất cả các lớp lại với nhau thành một CNN với đầu ra gồm các neural với số lượng tùy bài toán.



Hai lớp cuối cùng của các kết nối trong mạng là một lớp kết nối đầy đủ (fully connected layer). Lớp này nối tất cả các neuron của tầng trước với tất cả neuron của tầng ra.

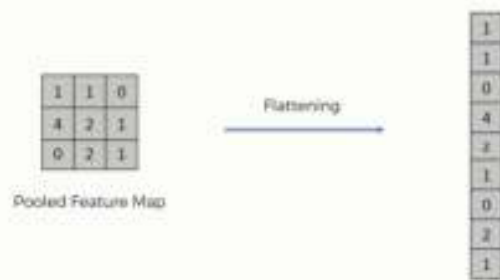
# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Fully connected layer

Sau khi ảnh được truyền qua nhiều convolutional layer và pooling layer thì mô hình đã học được tương đối các đặc điểm của ảnh (ví dụ mắt, mũi, khung mặt,...) các tensor của output layer cuối cùng có kích thước  $H*W*D$  sẽ được chuyển về 1 vector kích thước  $(H*W*D)$

Sau đó ta dùng các fully connected layer để kết hợp các đặc điểm của ảnh để cho ra output của mô hình.

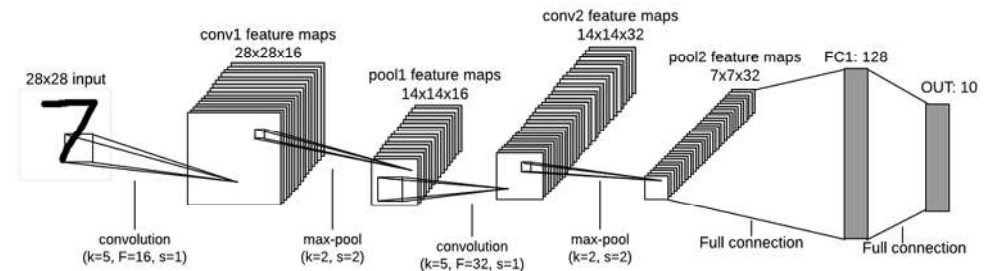


# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập: Mô hình convolutional neural network

Input image → Convolutional layer (Conv) + Pooling layer (Pool) → Fully connected layer (FC) → Output.



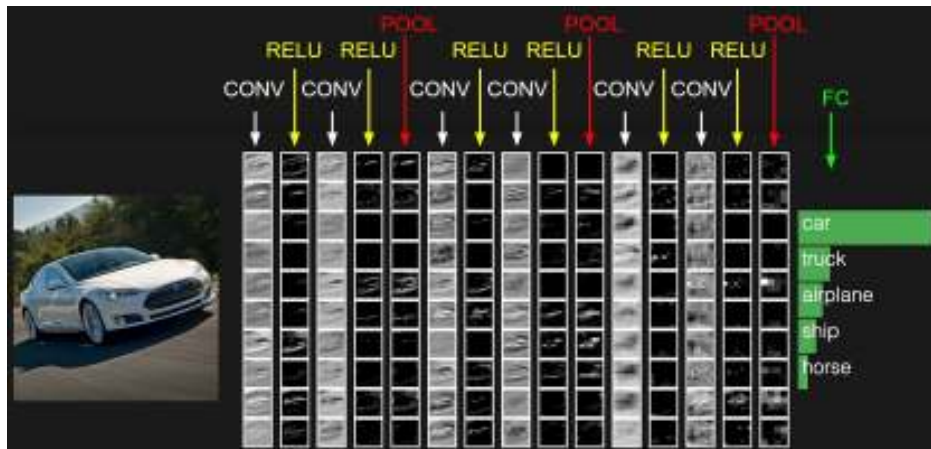
Ví dụ mô hình convolutional neural network



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Mạng neural tích chập:



Ví dụ mô hình convolutional neural network

# Thuộc tính của mô hình CNN

- ❖ Không gian tác động giữa các CNN (thông qua kernel có kích thước nhỏ)
  - Có ít tham số để học hơn
  - Cần ít nguồn để tính toán hơn
- ❖ Tham số chia sẻ (cùng một kernel được áp dụng cho tất cả đầu vào)
  - Duy trì khả năng phát hiện đặc trưng giống nhau cho ngõ vào
- ❖ Khả năng (tự động) **Học** cấu trúc cục bộ
- ❖ Có thể xử lý ngõ vào có kích thước thay đổi

## Ứng dụng của mô hình CNN

- ❖ Trong xử lý ảnh
  - Phân loại ảnh
  - Nhận dạng ảnh
  - Phân đoạn ảnh
  - Bám đối tượng
  - Nhận dạng điểm nổi bậc bằng thị giác
  - Nhận dạng gương mặt
- ❖ Mô bệnh học
- ❖ Xử lý giọng nói
- ❖ Tìm và nhận dạng ký tự văn bản (OCR)

## Ứng dụng của mô hình CNN

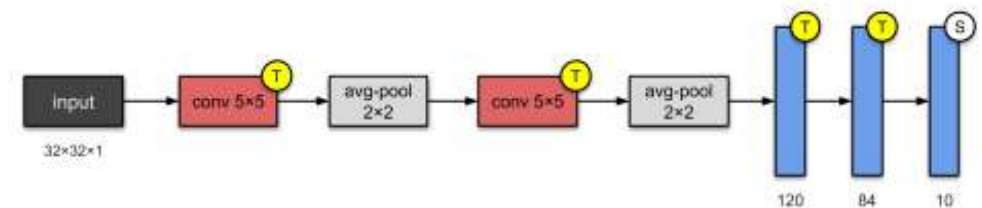
- ❖ Xử lý ngôn ngữ tự nhiên
- ❖ Nghiên cứu chế tạo thuốc
- ❖ Phân tích chuỗi thời gian
  - Đánh giá rủi ro sức khỏe và dấu hiệu sinh học của phát hiện lão hóa
  - Dự báo
  - Nhận dạng điện cơ (EMG)
- ❖ ...

## Các mô hình CNN thông dụng

- ❖ LeNet
- ❖ AlexNet
- ❖ VGG-16
- ❖ Inception-based architectures
- ❖ Xception
- ❖ ResNet

## Các mô hình CNN thông dụng

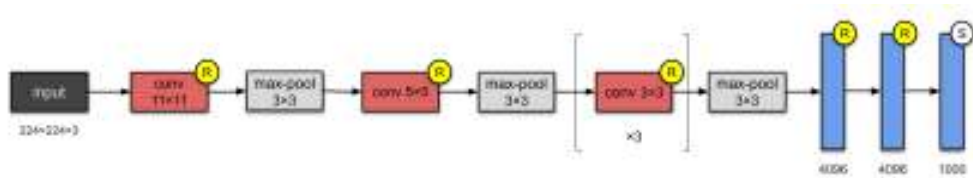
LeNet (1998)<sup>8</sup>



- Kiến trúc đã trở thành kiến trúc 'mẫu' tiêu chuẩn:
- (1) xếp chồng các tích chập và các lớp gộp (mức 7)
  - (2) kết thúc mạng một hoặc nhiều lớp fully - connected

## Các mô hình CNN thông dụng

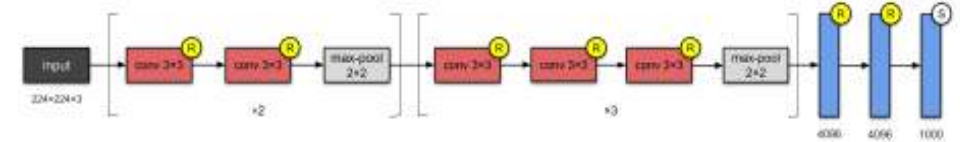
### AlexNet (2012)



- (1) Mạng sâu hơn và có nhiều bộ lọc hơn LeNet
  - (2) Sử dụng các lớp chập xếp chồng lên nhau
  - (3) Là công ty đầu tiên triển khai Rectified Linear Units (ReLU) làm hàm kích hoạt
- Năm 2012 đã vượt trội hơn tất cả các đối thủ cạnh tranh trước đó và giành chiến thắng trong thách thức ImageNet

## Các mô hình CNN thông dụng

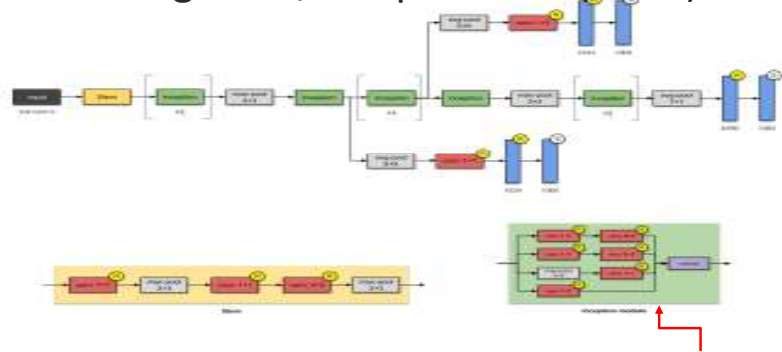
### VGG-16/VGG-19 (2014)<sup>10</sup>



- (1) mạng sâu hơn đáng kể so với các mạng trước đó (16 đến 19 lớp)
- (2) bộ lọc nhiều và nhỏ hơn (3x3) → Đứng thứ 2 trong thách thức ImageNet 2014

## Các mô hình CNN thông dụng

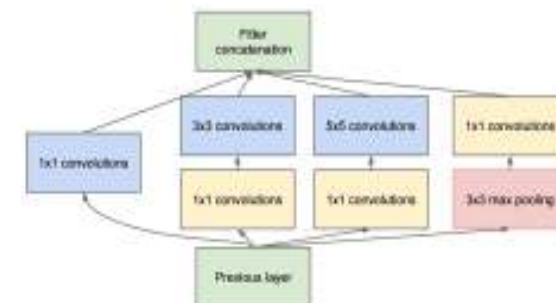
### GoogleNet/Inception- v1 (2014)



- (1) Mạng thậm chí còn sâu hơn (22 layers)
- (2) Đã triển khai module khởi tạo để giảm đáng kể số lượng tham số

## Các mô hình CNN thông dụng

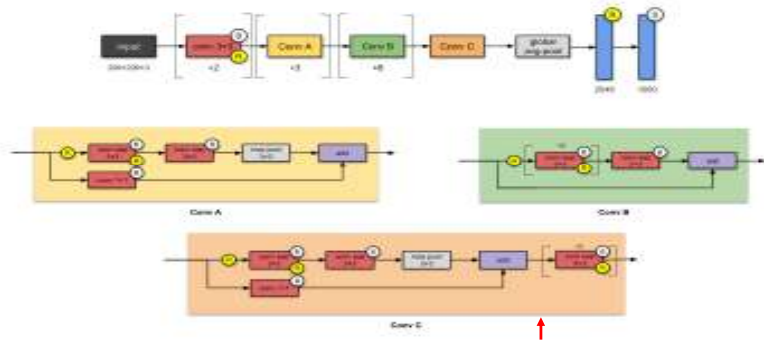
### Inception- 1 module



- Các cải tiến của module khởi động ban đầu (ví dụ: Inception-v3, Inception-v4) đã cải thiện hiệu suất của các mô hình được hỗ trợ khởi động, đáng chú ý nhất là bằng cách tái cấu trúc các tổ hợp lớn hơn thành các tổ hợp nhỏ hơn liên tiếp để học hơn

## Các mô hình CNN thông dụng

### Xception (2016)



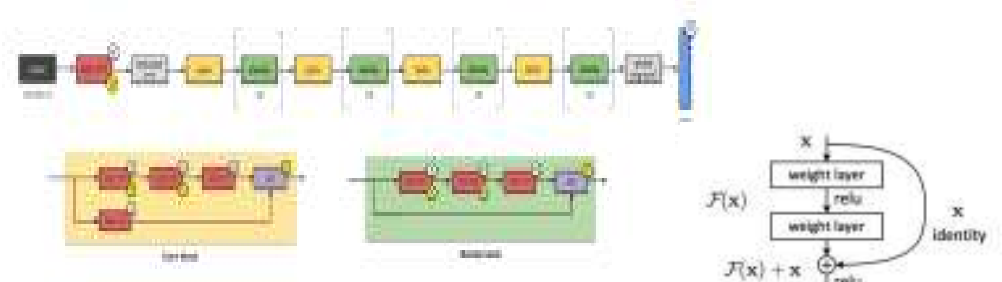
- (1) Các module khởi động được thay thế bằng các tích chập có thể phân tách theo chiều sâu

- ## (2) Cùng số lượng tham số như Inception

→ Vượt trội hơn một chút so với Inception v3 trên tập dữ liệu ImageNet và vượt trội hơn rất nhiều so với tập dữ liệu phân loại hình ảnh lớn hơn với 17.000 lớp

## Các mô hình CNN thông dụng

Resnet (2015)



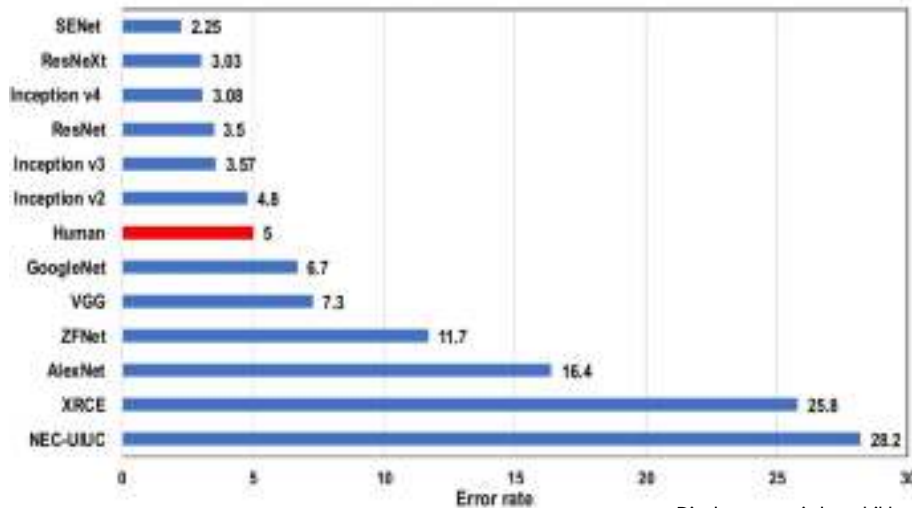
- (1) Thay vì học ánh xạ từ  $x$  đến  $H(x)$ , tìm học từ  $F(x) = H(x) - x \rightarrow$  bỏ qua việc tạo kết nối

- (2) Bỏ các các kết nối xếp chồng lên nhau

→ cho phép kiến trúc sâu hơn mà không làm giảm hiệu suất

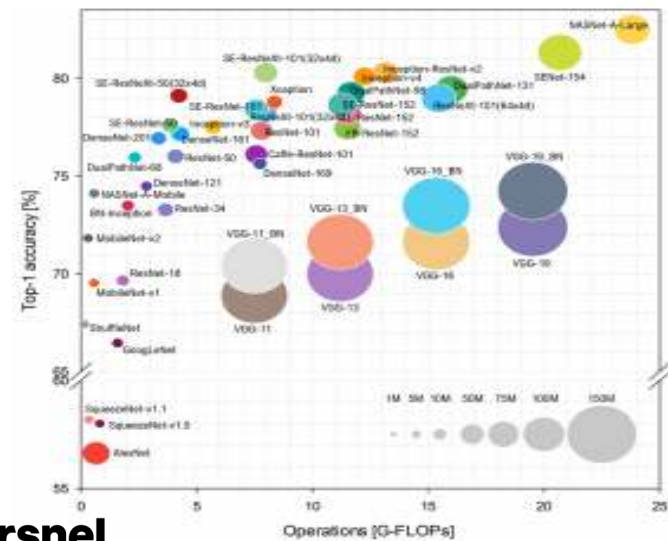
## Các mô hình CNN thông dụng

ImageNet Top 5 Error Rate



Dit document is beschikbaar op

## Các mô hình CNN thông dụng

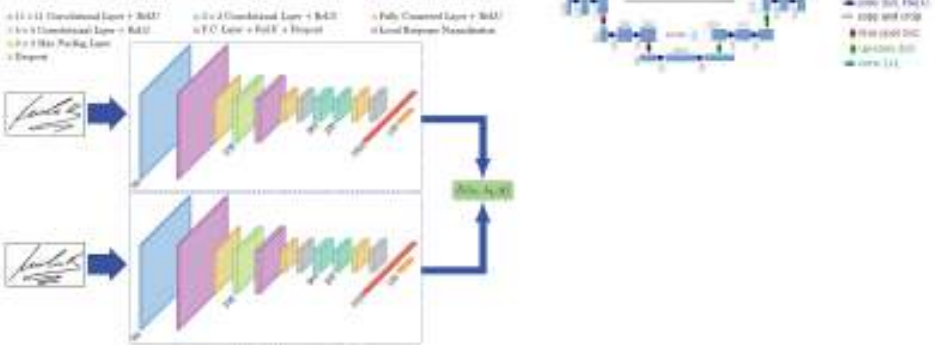


# Kiến trúc ứng dụng dựa trên mô hình CNN

❖ UNET: semantic segmentation

❖ Siamese Network:

- Học rất ít dữ liệu



# Đối tượng dễ nhầm lẫn?

Mô hình CNN có thể dễ dàng bị đánh lừa

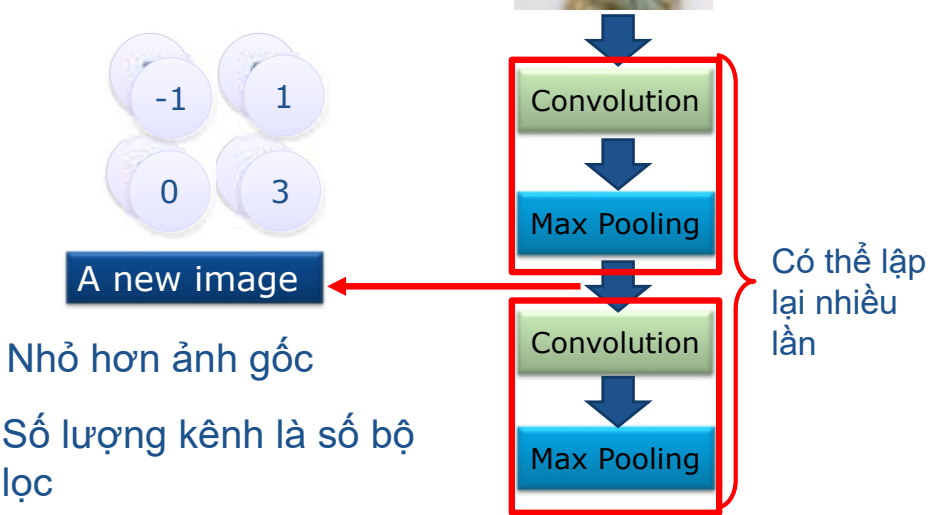


# Đối tượng dễ nhầm lẫn?

CNN không tốt trong việc khái quát hóa về tỷ lệ, dịch chuyển và xoay trong hình, cũng như các góc nhìn ba chiều khác nhau.



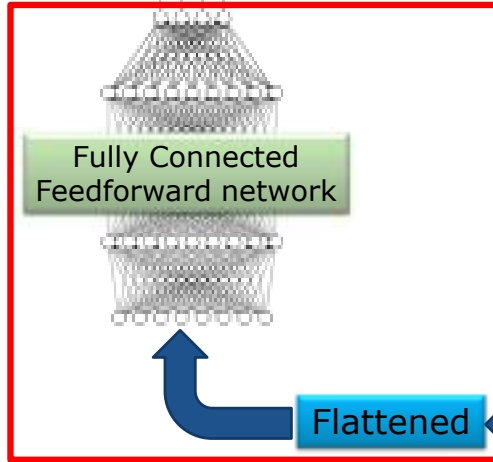
# Các bước của CNN



## Các bước của CNN



cat dog .....



Convolution

Max Pooling

A new image

Convolution

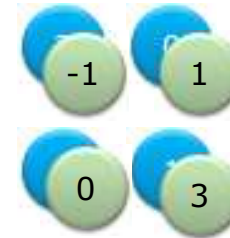
Max Pooling

A new image

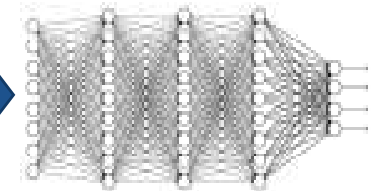
Flattened

## 3 Các bước của CNN

Flattening (làm phẳng)



Flattened

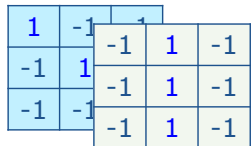


Fully Connected Feedforward network

## CNN in Keras

Only modified the **network structure** and **input format (vector → 3-D tensor)**

```
model2.add( Convolution2D( 25,3,3,
                           input_shape=(28,28,1) ) )
```

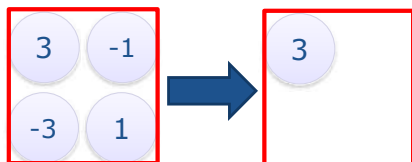


There are 25 3x3 filters.

Input\_shape = ( 28 , 28 , 1 )

28 x 28 pixels 1: black/white, 3: RGB

```
model2.add( MaxPooling2D( (2,2) ) )
```



input

Convolution

Max Pooling

Convolution

Max Pooling

## CNN in Keras

Only modified the **network structure** and **input format (vector → 3-D array)**

```
model2.add( Convolution2D( 25,3,3,
                           input_shape=(28,28,1) ) )
```

How many parameters for each filter? 9

1 x 28 x 28

25 x 26 x 26

```
model2.add( MaxPooling2D( (2,2) ) )
```

25 x 13 x 13

```
model2.add( Convolution2D( 50,3,3,
                           input_shape=(28,28,1) ) )
```

How many parameters for each filter? 225= 25x9

50 x 11 x 11

```
model2.add( MaxPooling2D( (2,2) ) )
```

50 x 5 x 5

Input

Convolution

Max Pooling

Convolution

Max Pooling



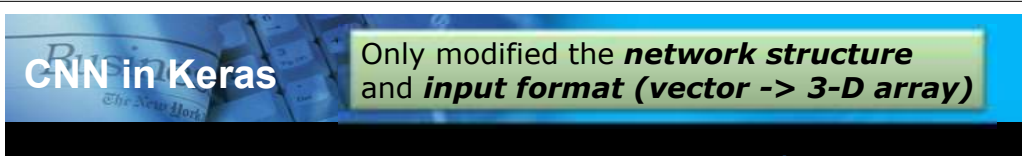
CNN in Keras

Only modified the **network structure** and **input format (vector -> 3-D array)**

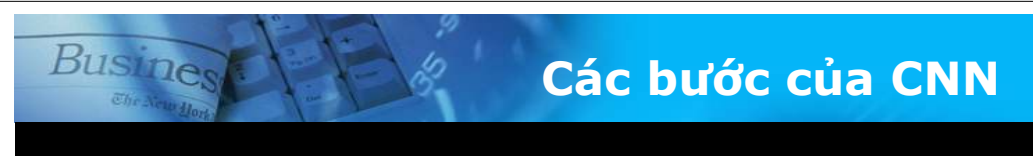


CNN in Keras

Only modified the **network structure** and **input format (vector -> 3-D array)**



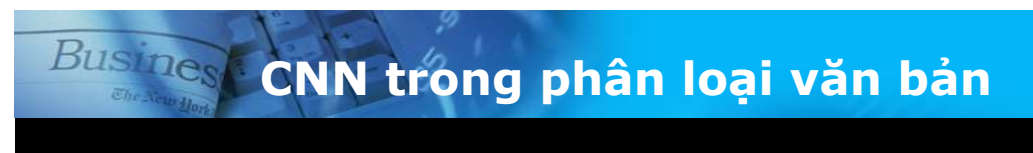
**Các bước của CNN**



**CNN trong nhận dạng giọng nói**



**CNN trong phân loại văn bản**



# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Cách chọn tham số cho CNN

1. Số các convolution layer: càng nhiều các convolution layer thì quá trình thực hiện càng được cải thiện. Sau khoảng 3 hoặc 4 layer, các tác động được giảm một cách đáng kể
2. Filter size: thường filter theo kích thước  $5 \times 5$  hoặc  $3 \times 3$
3. Pooling size: thường là  $2 \times 2$  hoặc  $4 \times 4$  cho ảnh đầu vào lớn
4. Cách cuối cùng là thực hiện nhiều lần việc train test để chọn ra được tham số tốt nhất.



## Ví dụ hoạt động Convolution

Giả sử matrix **M**  $6 \times 6$  là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* là 1 matrix **CM**  $4 \times 4$  (còn được gọi là *feature map*) được tính như sau:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 4 | 7 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

**M**

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

**F**

↓  
convolution operator

=

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

**CM**

## Ví dụ hoạt động Convolution

Giả sử matrix **M**  $6 \times 6$  là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM**  $4 \times 4$  (còn được gọi là *feature map*) được tính như sau:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 4 | 7 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

**M**

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

**F**

↓

=

|    |  |  |  |
|----|--|--|--|
| -5 |  |  |  |
|    |  |  |  |
|    |  |  |  |
|    |  |  |  |

**CM**

$$3 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 0 \cdot 0 + 5 \cdot 0 + 7 \cdot 0 + 1 \cdot (-1) + 8 \cdot (-1) + 2 \cdot (-1)$$

## Ví dụ hoạt động Convolution

Giả sử matrix **M**  $6 \times 6$  là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM**  $4 \times 4$  (còn được gọi là *feature map*) được tính như sau:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 4 | 7 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

**M**

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

**F**

↓

=

|    |    |  |  |
|----|----|--|--|
| -5 | -4 |  |  |
|    |    |  |  |
|    |    |  |  |
|    |    |  |  |

**CM**

$$0 \cdot 1 + 5 \cdot 1 + 7 \cdot 1 + 1 \cdot 0 + 8 \cdot 0 + 2 \cdot 0 + 2 \cdot (-1) + 9 \cdot (-1) + 5 \cdot (-1)$$

## Ví dụ hoạt động Convolution

Giả sử matrix **M** 6x6 là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM** 4x4 (còn được gọi là *feature map*) được tính như sau:

$$\begin{array}{|c|c|c|c|c|c|} \hline 3 & 0 & 1 & 2 & 4 & 7 \\ \hline 1 & 5 & 8 & 9 & 3 & 1 \\ \hline 2 & 7 & 2 & 5 & 1 & 3 \\ \hline 0 & 1 & 3 & 1 & 7 & 8 \\ \hline 4 & 2 & 1 & 6 & 2 & 8 \\ \hline 2 & 4 & 5 & 2 & 3 & 9 \\ \hline \end{array} \quad * \quad \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline -5 & -4 & 0 & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array}$$

**M** **F** **CM**

$$1*1 + 8*1 + 2*1 + 2*0 + 9*0 + 5*0 + 4*(-1) + 3*(-1) + 1*(-1)$$

## Ví dụ hoạt động Convolution

Giả sử matrix **M** 6x6 là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM** 4x4 (còn được gọi là *feature map*) được tính như sau:

$$\begin{array}{|c|c|c|c|c|c|} \hline 3 & 0 & 1 & 2 & 4 & 7 \\ \hline 1 & 5 & 8 & 9 & 3 & 1 \\ \hline 2 & 7 & 2 & 5 & 1 & 3 \\ \hline 0 & 1 & 3 & 1 & 7 & 8 \\ \hline 4 & 2 & 1 & 6 & 2 & 8 \\ \hline 2 & 4 & 5 & 2 & 3 & 9 \\ \hline \end{array} \quad * \quad \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline -5 & -4 & 0 & 8 \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array}$$

**M** **F** **CM**

$$2*1 + 9*1 + 5*1 + 4*0 + 3*0 + 1*0 + 7*(-1) + 1*(-1) + 3*(-1)$$

## Ví dụ hoạt động Convolution

Giả sử matrix **M** 6x6 là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM** 4x4 (còn được gọi là *feature map*) được tính như sau:

$$\begin{array}{|c|c|c|c|c|c|} \hline 3 & 0 & 1 & 2 & 4 & 7 \\ \hline 1 & 5 & 8 & 9 & 3 & 1 \\ \hline 2 & 7 & 2 & 5 & 1 & 3 \\ \hline 0 & 1 & 3 & 1 & 7 & 8 \\ \hline 4 & 2 & 1 & 6 & 2 & 8 \\ \hline 2 & 4 & 5 & 2 & 3 & 9 \\ \hline \end{array} \quad * \quad \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline -5 & -4 & 0 & 8 \\ \hline -10 & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array}$$

**M** **F** **CM**

$$1*1 + 2*1 + 0*1 + 5*0 + 7*0 + 1*0 + 8*(-1) + 2*(-1) + 3*(-1)$$

## Ví dụ hoạt động Convolution

Giả sử matrix **M** 6x6 là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM** 4x4 (còn được gọi là *feature map*) được tính như sau:

$$\begin{array}{|c|c|c|c|c|c|} \hline 3 & 0 & 1 & 2 & 4 & 7 \\ \hline 1 & 5 & 8 & 9 & 3 & 1 \\ \hline 2 & 7 & 2 & 5 & 1 & 3 \\ \hline 0 & 1 & 3 & 1 & 7 & 8 \\ \hline 4 & 2 & 1 & 6 & 2 & 8 \\ \hline 2 & 4 & 5 & 2 & 3 & 9 \\ \hline \end{array} \quad * \quad \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline -5 & -4 & 0 & 8 \\ \hline -10 & -2 & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array}$$

**M** **F** **CM**

$$5*1 + 7*1 + 1*1 + 8*0 + 2*0 + 3*0 + 9*(-1) + 5*(-1) + 1*(-1)$$

## Ví dụ hoạt động Convolution

Giả sử matrix **M** 6x6 là ma trận vào. Convolution 2 chiều của **M** bằng lọc (hay *kernel*) **F** và *stride* bằng 1 matrix **CM** 4x4 (còn được gọi là *feature map*) được tính như sau:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 4 | 7 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | ↓ | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

**M**

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

**F**

=

|     |    |    |     |
|-----|----|----|-----|
| -5  | -4 | 0  | 8   |
| -10 | -2 | 2  | 3   |
| 0   | -2 | -4 | -7  |
| -3  | -2 | -3 | -16 |

**CM**

$$1*1 + 6*1 + 2*1 + 7*0 + 2*0 + 3*0 + 8*(-1) + 8*(-1) + 9*(-1)$$

## Ví dụ khi Stride lớn

Giả sử **M** là matrix 7x7. Convolution 2D của **M** bằng lọc **F** và *stride* bằng 2 là 1 matrix **CM** 3x3 được tính như sau:

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 3 | 7 | 4 | 6 | 2 | 9 |
| 6 | 6 | 9 | 8 | 7 | 4 | 3 |
| 3 | 4 | 8 | 3 | 8 | 9 | 7 |
| 7 | 8 | 3 | 6 | 6 | 3 | 4 |
| 4 | 2 | 1 | 8 | 3 | 4 | 6 |
| 3 | 2 | 4 | 1 | 9 | 8 | 3 |
| 0 | 1 | 3 | 9 | 2 | 1 | 4 |

**M**

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

**F**

=

|    |  |  |
|----|--|--|
| 91 |  |  |
|    |  |  |
|    |  |  |

**CM**

$$2*3 + 6*1 + 3*(-1) + 3*4 + 6*0 + 4*0 + 7*4 + 9*2 + 8*3$$

## Ví dụ khi Stride lớn

Giả sử **M** là matrix 7x7. Convolution 2D của **M** bằng lọc **F** và *stride* bằng 2 là 1 matrix **CM** 3x3 được tính như sau:

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 3 | 7 | 4 | 6 | 2 | 9 |
| 6 | 6 | 9 | 8 | 7 | 4 | 3 |
| 3 | 4 | 8 | 3 | 8 | 9 | 7 |
| 7 | 8 | 3 | 6 | 6 | 3 | 4 |
| 4 | 2 | 1 | 8 | 3 | 4 | 6 |
| 3 | 2 | 4 | 1 | 9 | 8 | 3 |
| 0 | 1 | 3 | 9 | 2 | 1 | 4 |

**M**

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

**F**

=

|    |     |  |
|----|-----|--|
| 91 | 100 |  |
|    |     |  |
|    |     |  |

**CM**

$$7*3 + 9*1 + 8*(-1) + 4*4 + 8*0 + 3*0 + 6*4 + 7*2 + 8*3$$

## Ví dụ khi Stride lớn

Giả sử **M** là matrix 7x7. Convolution 2D của **M** bằng lọc **F** và *stride* bằng 2 là 1 matrix **CM** 3x3 được tính như sau:

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 3 | 7 | 4 | 6 | 2 | 9 |
| 6 | 6 | 9 | 8 | 7 | 4 | 3 |
| 3 | 4 | 8 | 3 | 8 | 9 | 7 |
| 7 | 8 | 3 | 6 | 6 | 3 | 4 |
| 4 | 2 | 1 | 8 | 3 | 4 | 6 |
| 3 | 2 | 4 | 1 | 9 | 8 | 3 |
| 0 | 1 | 3 | 9 | 2 | 1 | 4 |

**M**

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

**F**

=

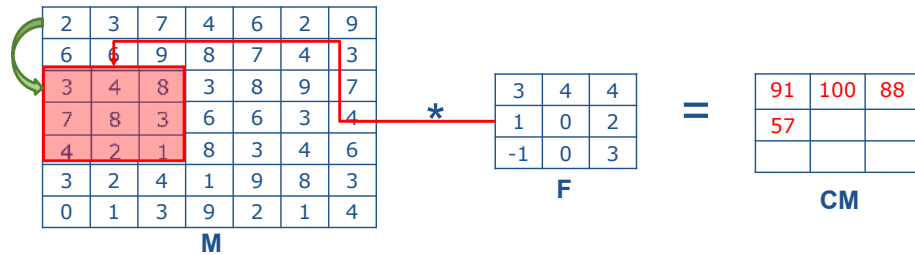
|    |     |    |
|----|-----|----|
| 91 | 100 | 88 |
|    |     |    |
|    |     |    |

**CM**

$$6*3 + 7*1 + 8*(-1) + 2*4 + 4*0 + 9*0 + 9*4 + 3*2 + 7*3$$

## Ví dụ khi Stride lớn

Giả sử  $M$  là matrix  $7 \times 7$ . Convolution 2D của  $M$  bằng lọc  $F$  và **stride** bằng **2** là 1 matrix  $CM$   $3 \times 3$  được tính như sau:



$$3 \times 3 + 7 \times 1 + 4 \times (-1) + 4 \times 4 + 8 \times 0 + 2 \times 0 + 8 \times 4 + 3 \times 2 + 1 \times 3$$

## Ví dụ convolution và sải bước

Các tham số của mạng cần học

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

6 x 6 image

|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |

Filter 1

|    |   |    |
|----|---|----|
| -1 | 1 | -1 |
| -1 | 1 | -1 |
| -1 | 1 | -1 |

Filter 2

⋮

Mỗi bộ lọc phát hiện một mẫu nhỏ ( $3 \times 3$ ).

## Ví dụ convolution và sải bước

stride=1

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

6 x 6 image

Dot product

→

3

-1

|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |

Filter 1

## Ví dụ convolution và sải bước

If stride=2

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |

6 x 6 image

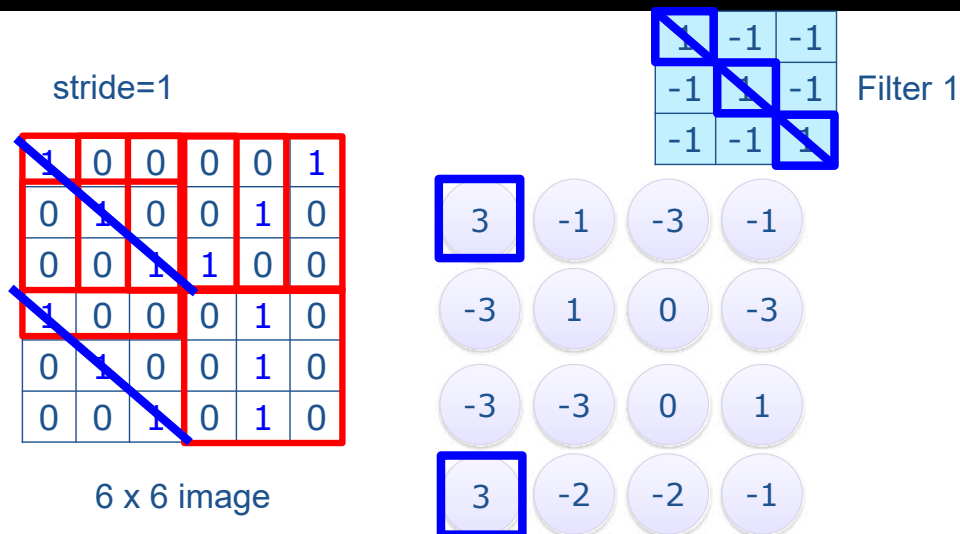
|    |    |    |
|----|----|----|
| 1  | -1 | -1 |
| -1 | 1  | -1 |
| -1 | -1 | 1  |

Filter 1

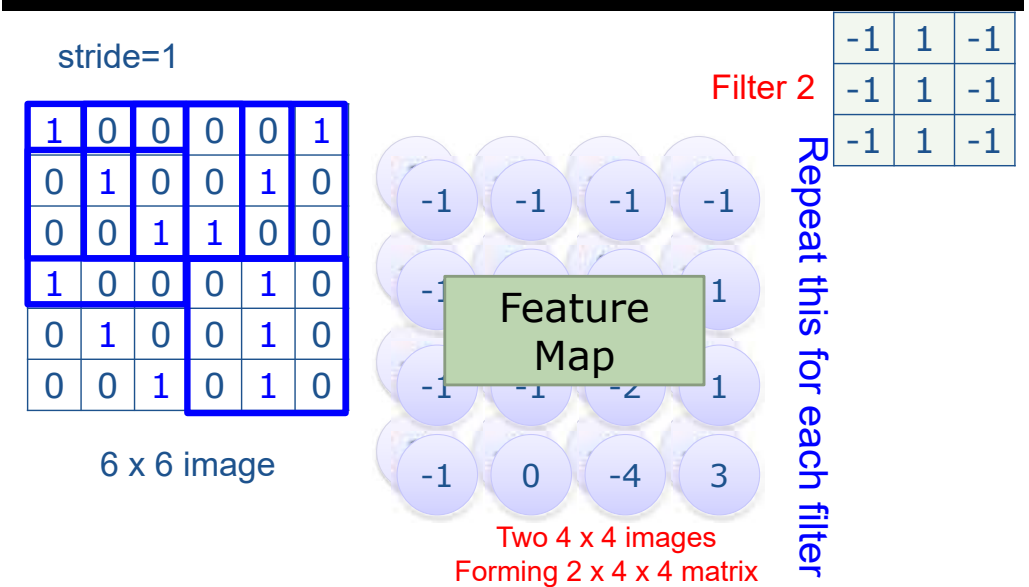
3

-3

## Ví dụ convolution và sải bước



## Ví dụ convolution và sải bước

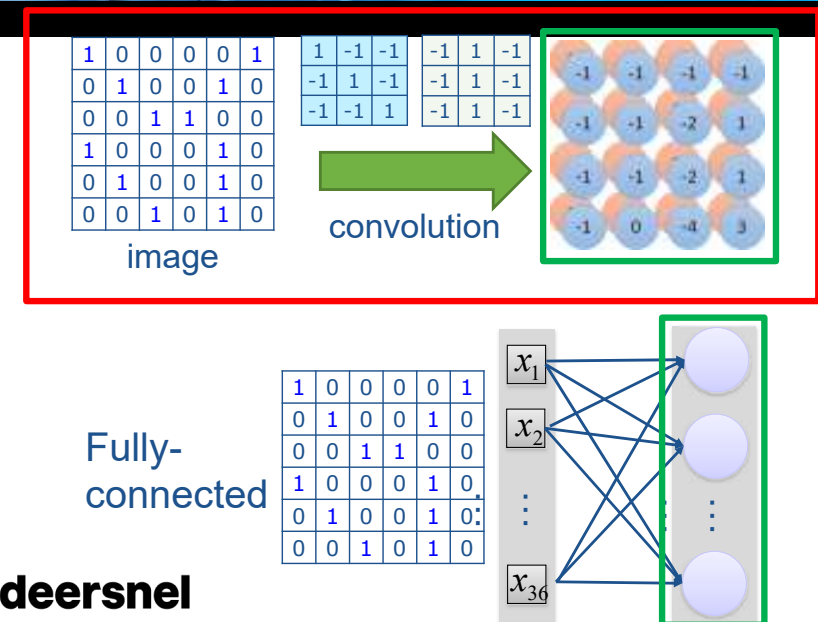


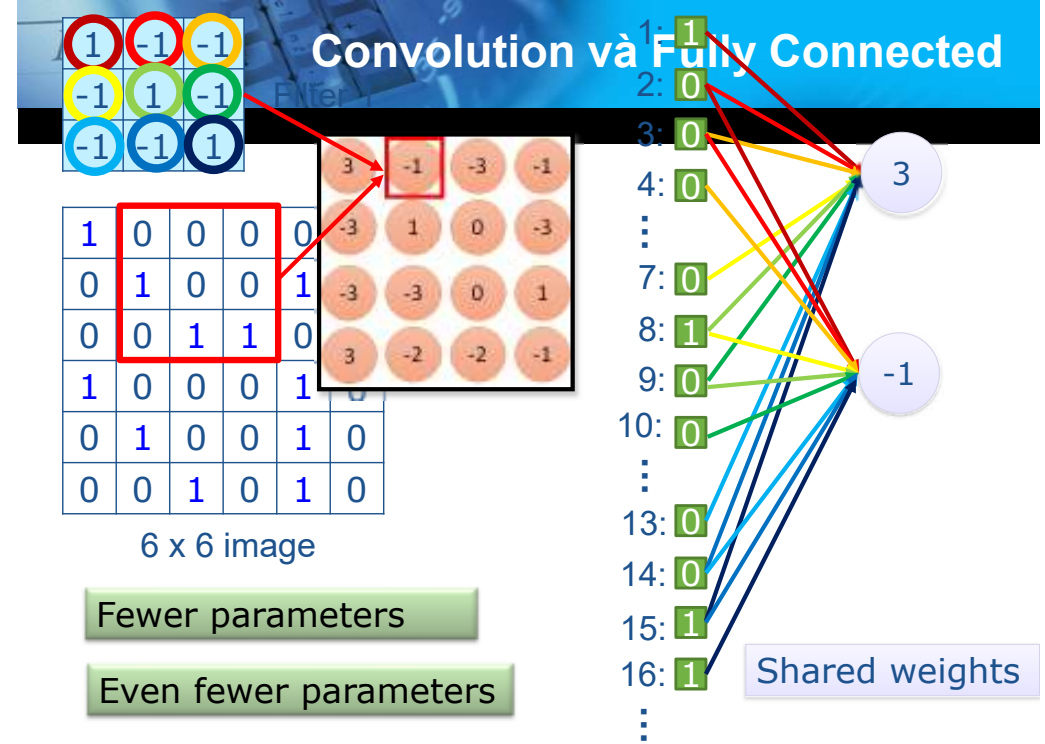
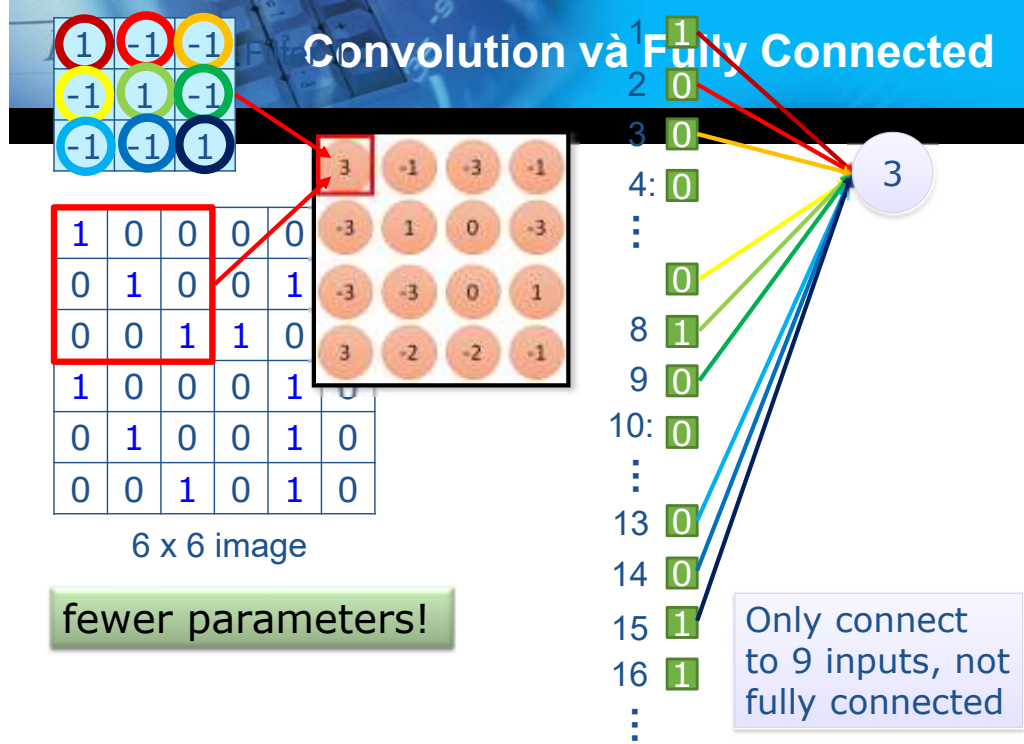
## Convolution và Fully Connected

**CNN nén bằng một mạng fully connected được thực hiện theo hai cách:**

- ❖ Giảm số lượng kết nối, trọng số chia sẻ trên các cạnh
- ❖ Max pooling làm giảm đi độ phức tạp

## Convolution và Fully Connected





## Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

### Bài tập mạng CNN cho ảnh xám

Cho ảnh u sau:

|     |     |     |     |
|-----|-----|-----|-----|
| 90  | 150 | 59  | 125 |
| 50  | 185 | 125 | 225 |
| 85  | 95  | 200 | 59  |
| 150 | 45  | 150 | 225 |

a. Chọn ngưỡng  $\delta = 100$ , chuyển ảnh u sang ảnh nhị phân  $g(x, y) = \begin{cases} 1, u(x, y) \geq \delta \\ 0 \text{ else} \end{cases}$

b. Dùng mạng CNN có cấu trúc sau:

• Lớp chập 1:

- Đệm và sải bước:  $p = 1, s = 1$
- Với 2 Kernel 3x3, hàm tác động là tuyến tính

$$W_1 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}, W_2 = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

• Lớp gộp 1:

- Đệm và sải bước:  $p=0, s=2$
- Max-pooling 2x2

Downloaded by Quan Dang (dangquan3172006@gmail.com)

## Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

### Bài tập mạng CNN cho ảnh xám

- Lớp chập 2:
  - Đệm và sải bước:  $p = 1, s = 1$
  - Với 1 Kernel 3x3, hàm tác động là tuyến tính
$$\text{Kernel} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
- Lớp gộp 2:
  - Đệm và sải bước:  $p=0, s=1$
  - Max-pooling 2x2
$$\text{Kernel} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Bài tập mạng CNN cho ảnh xám

- Thông số mạng truyền thẳng:

$$V(0) = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, W(0) = [0 \quad 0 \quad 1], \sigma = 0.1,$$

$$a_o(net) = net, a_h(net) = \tanh(net)$$

Tính trọng số mạng được cập nhật ở bước tiếp theo?

- $V(1)$
- $W(1)$

# Mạng CNN (Convolutional neural network)

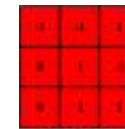
www.tvhungdhcn@gmail

## Bài tập mạng CNN cho ảnh màu

Cho ảnh màu RGB sau:

|   | R  |    |    |    |    |    |    |    |   | G  |    |    |    |    |    |    |    |   | B  |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|---|----|----|----|----|----|----|----|----|
|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 0 | 10 | 10 | 20 | 20 | 20 | 24 | 24 | 24 | 0 | 52 | 52 | 57 | 55 | 56 | 52 | 51 | 52 | 0 | 12 | 12 | 40 | 40 | 20 | 20 | 20 | 20 |
| 1 | 10 | 10 | 10 | 20 | 20 | 24 | 24 | 24 | 1 | 50 | 49 | 51 | 50 | 52 | 53 | 58 | 50 | 1 | 12 | 12 | 40 | 40 | 20 | 20 | 20 | 20 |
| 2 | 10 | 10 | 10 | 20 | 20 | 23 | 24 | 24 | 2 | 51 | 51 | 52 | 52 | 56 | 57 | 60 | 51 | 2 | 10 | 40 | 40 | 10 | 10 | 20 | 20 | 20 |
| 3 | 20 | 20 | 20 | 20 | 20 | 23 | 23 | 23 | 3 | 48 | 50 | 51 | 49 | 53 | 59 | 63 | 48 | 3 | 10 | 10 | 10 | 10 | 10 | 20 | 20 | 20 |
| 4 | 20 | 20 | 80 | 80 | 23 | 23 | 23 | 23 | 4 | 49 | 51 | 52 | 55 | 58 | 64 | 67 | 49 | 4 | 30 | 30 | 30 | 20 | 20 | 20 | 40 | 40 |
| 5 | 20 | 80 | 80 | 80 | 14 | 14 | 14 | 23 | 5 | 14 | 15 | 15 | 16 | 16 | 16 | 17 | 14 | 5 | 30 | 30 | 20 | 20 | 20 | 40 | 40 | 40 |
| 6 | 80 | 80 | 80 | 80 | 14 | 14 | 14 | 14 | 6 | 51 | 51 | 52 | 52 | 56 | 57 | 60 | 51 | 6 | 30 | 30 | 30 | 30 | 40 | 40 | 40 | 40 |
| 7 | 10 | 10 | 20 | 20 | 20 | 24 | 24 | 24 | 7 | 48 | 50 | 51 | 49 | 53 | 59 | 63 | 48 | 7 | 30 | 30 | 30 | 30 | 40 | 40 | 40 | 40 |

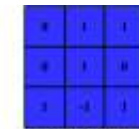
Các phần tử cấu trúc tương ứng của ba kênh màu RGB như sau



Kernel kênh R



Kernel kênh G



Kernel kênh B

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Bài tập mạng CNN cho ảnh màu

- a. Lớp tích chập: Cho hệ số bias = 1.

cho ảnh tương ứng với 3 phần tử cấu trúc RGB

Với 3 Kernel size = (3,3), stride=1, padding=0

| R       | G       | B      |
|---------|---------|--------|
| -1 -1 1 | 1 0 0   | 0 1 1  |
| 0 1 -1  | 1 -1 -1 | 0 1 0  |
| 0 1 1   | 1 0 -1  | 1 -1 1 |

- b. Lớp gộp:

Tính average pooling layer size = (2,2), stride=2, padding=1 đặt tên ảnh ra là  $u$

$$\text{Kernel} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

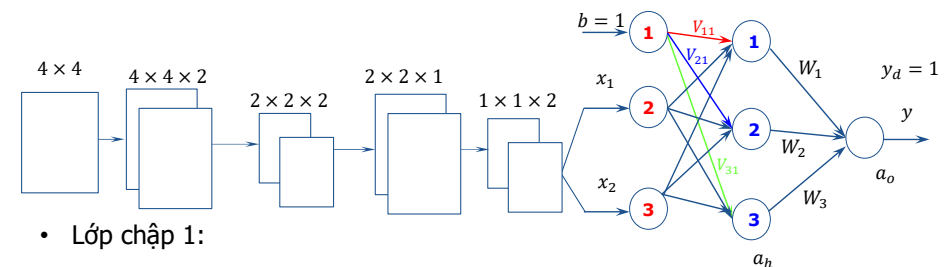
- c. Chọn ngưỡng  $\delta = 100$ , chuyển ảnh  $u$  sang ảnh nhị phân  $g(x, y) = \begin{cases} 1, u(x, y) \geq \delta \\ 0 \text{ else} \end{cases}$

# Mạng CNN (Convolutional neural network)

www.tvhungdhcn@gmail

## Bài tập mạng CNN cho ảnh màu

- d. Từ dữ liệu ảnh  $g(x, y)$  được ở câu c, sử dụng mạng CNN có cấu trúc sau:



- Lớp chập 1:

– Đệm và sải bước:  $p = 1, s = 1$

– Với 2 Kernel 3x3, hàm tác động là tuyến tính

$$W_1 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}, W_2 = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

- Lớp gộp 1:

– Đệm và sải bước:  $p=0, s=2$

– Max-pooling 2x2

# Mạng CNN (Convolutional neural network)

[www.tvhungdhcn@gmail](http://www.tvhungdhcn@gmail)

## Bài tập mạng CNN cho ảnh màu

- Lớp chập 2:
  - Đệm và sải bước:  $p = 1, s = 1$
  - Với 1 Kernel 3x3, hàm tác động là tuyến tính

$$\text{Kernel} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Lớp gộp 2:
  - Đệm và sải bước:  $p=0, s=1$
  - Max-pooling 2x2

$$\text{Kernel} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

# Mạng CNN (Convolutional neural network)

[www.tvhungdhcn@gmail](http://www.tvhungdhcn@gmail)

## Bài tập mạng CNN cho ảnh màu

- Thông số mạng truyền thẳng:

$$V(0) = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, W(0) = [0 \quad 0 \quad 1], \sigma = 0.1,$$

$$a_o(net) = net, a_h(net) = \tanh(net)$$

Tính trọng số mạng được cập nhật ở bước tiếp theo?

- $V(1)$
- $W(1)$

# Thank You !

[www.tvhungdhcn@gmail.com](http://www.tvhungdhcn@gmail.com)

